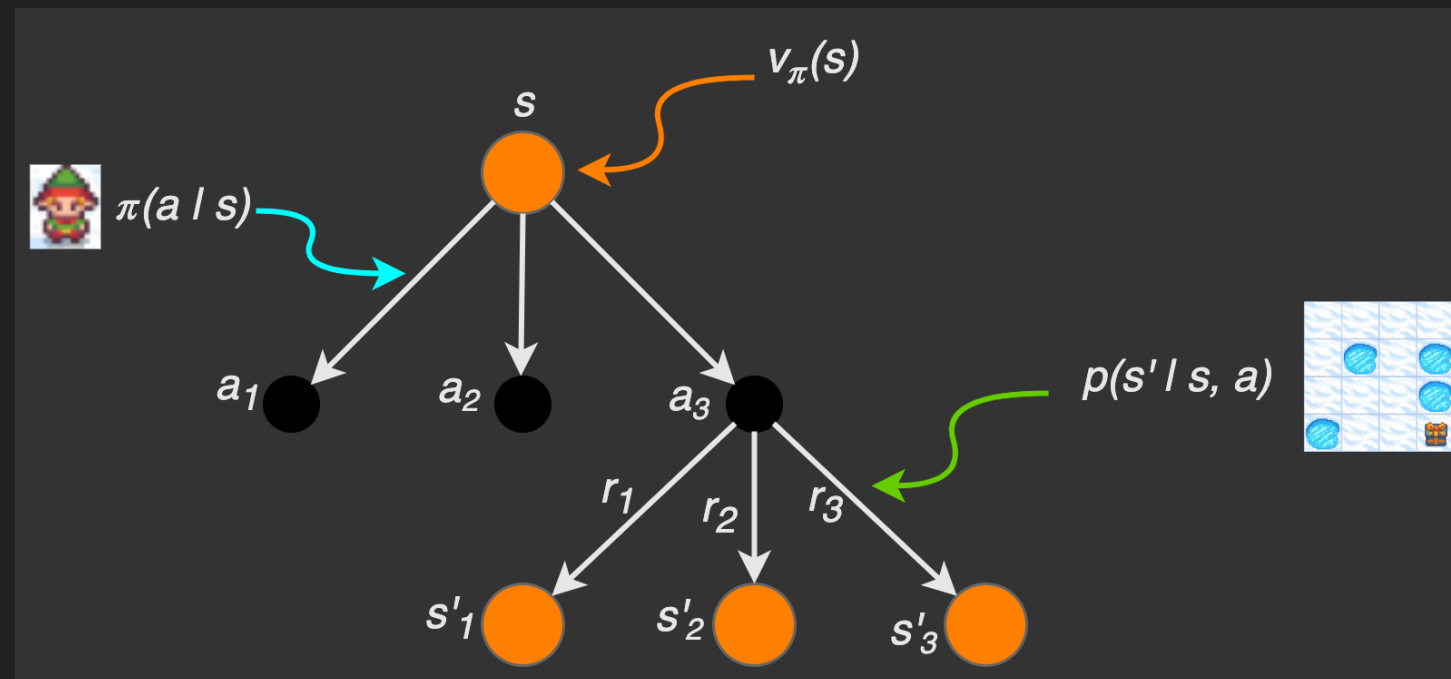


3

PROGRAMACIÓN DINÁMICA

RESUMEN



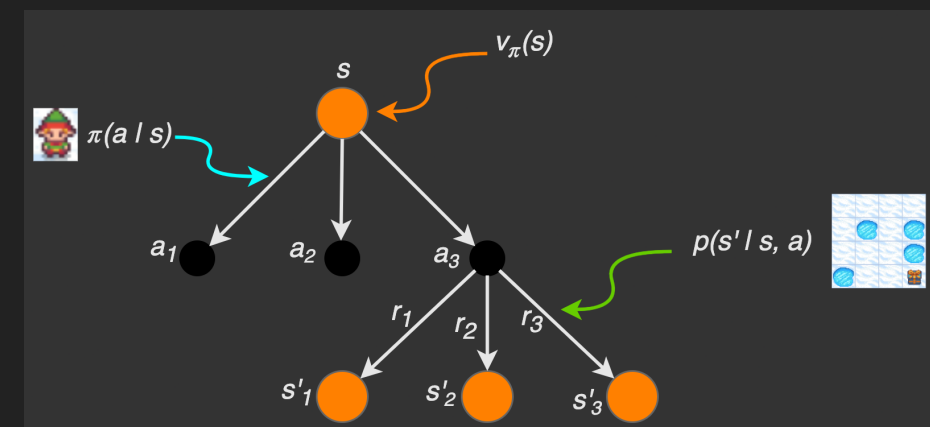
- ▶ Política (π) $\rightarrow$ Pertenece al AGENTE
- ▶ Probabilidad de Transición de Estado (p) $\rightarrow$ pertenece al ENTORNO

POLÍTICA

- Política ($\pi|s$): es la probabilidad de que el agente elija la acción a cuando se encuentra en el estado s .
- Ejemplo: En un semáforo en amarillo (s), la política de un vehículo autónomo podría ser:

$$\pi(\text{acelerar} \mid \text{amarillo}) = 0.3$$

$$\pi(\text{frenar} \mid \text{amarillo}) = 0.7$$



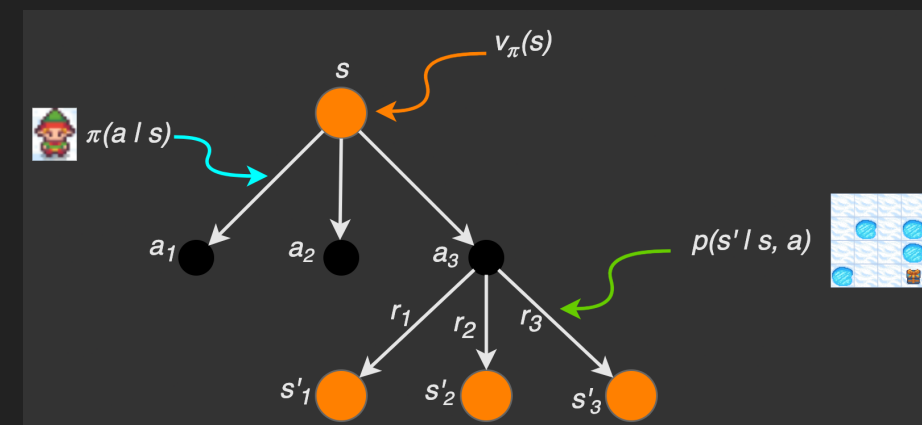
PROBABILIDAD DE TRANSICIÓN

- Probabilidad de Transición de Estado (p): $P(s'|s, a)$ es la probabilidad de que el entorno mueva al agente al estado s' dado que estaba en el estado s y tomó la acción a .
- Ejemplo: Si un vehículo está en el estado "suelo seco" (s) y toma la acción "frenar" (a), la probabilidad de llegar al estado "quieto" (s') es muy alta:

$$P(\text{quieto} \mid \text{suelo seco, frenar}) = 0.99$$

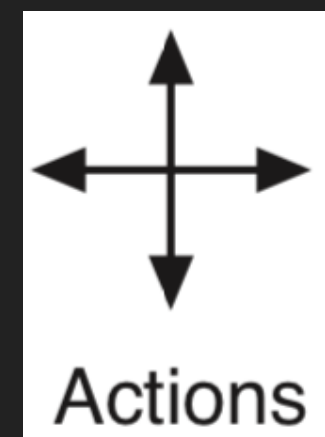
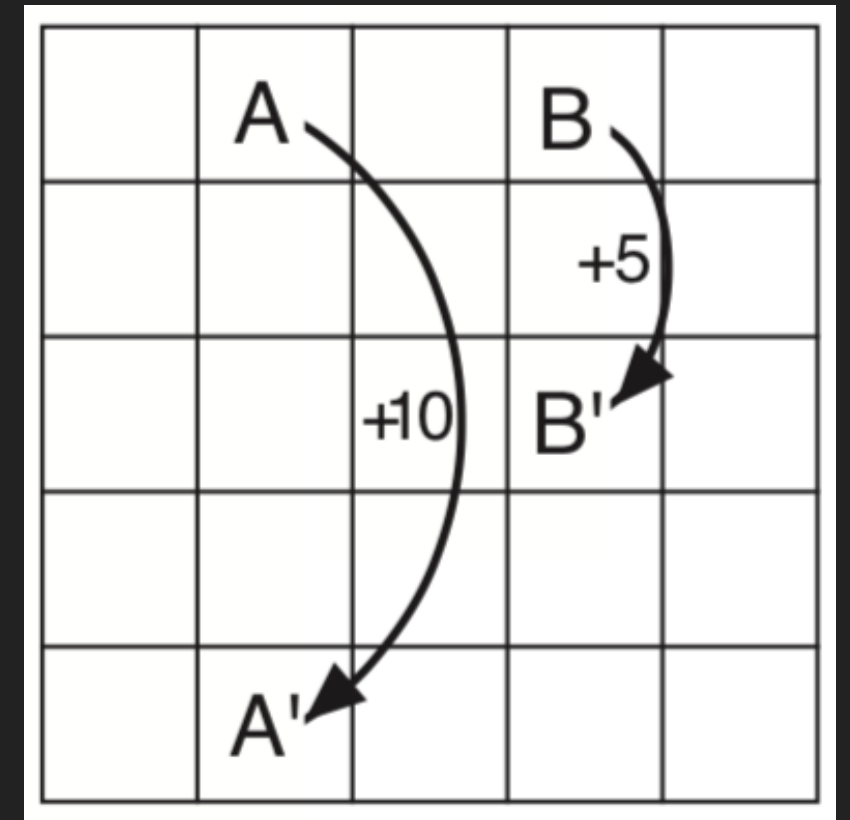
Pero si está en el estado "suelo con hielo" (s) y toma la misma acción "frenar" (a), la probabilidad de terminar "patinando" (s') es alta:

$$P(\text{patinando} \mid \text{suelo con hielo, frenar}) = 0.8$$



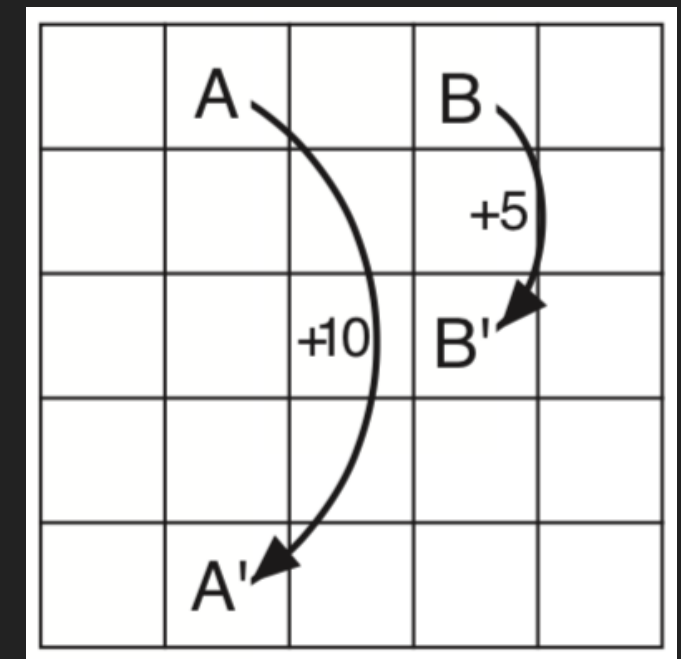
EJEMPLO GRIDWORLD 5X5

- ▶ La figura muestra una representación rectangular de un Gridworld de un MDP finito simple.
- ▶ Acciones posibles: **norte, sur, este y oeste**
- ▶ Recompensas:
 - ✓ **-1** por chocar con los bordes.
 - ✓ **0** por pasar de una celda a otra.
 - ✓ **+10** por moverse desde A.
 - ✓ **+5** por moverse desde B.



EJEMPLO GRIDWORLD 5X5

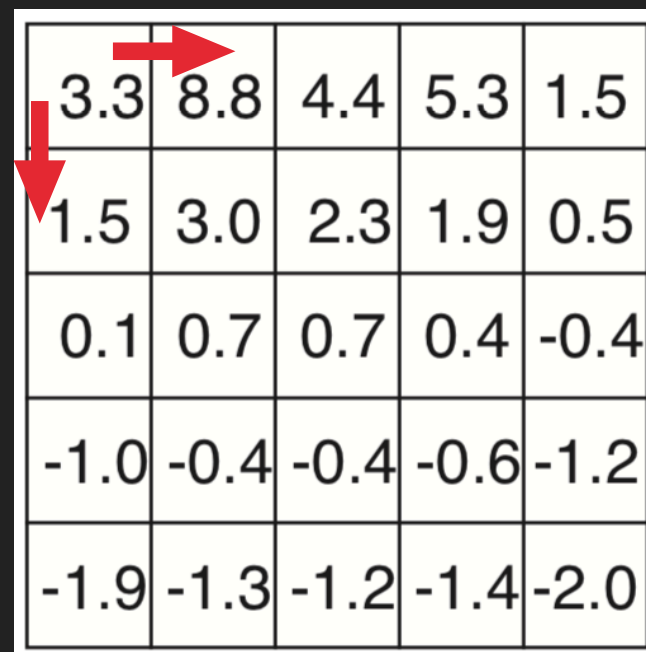
- Supongamos que el agente selecciona todas las acciones con **igual probabilidad** en todos los estados.
- La figura inferior muestra la **función de valor de estado $v_{\pi}(s)$** , para esta política, en el caso de recompensa descontada con $\gamma=0.9$.
- Los valores negativos cerca del borde inferior son el resultado de la **alta probabilidad de chocar contra el borde de la cuadrícula bajo la política aleatoria**.
- El estado A es el mejor estado en el que estar bajo esta política, **pero su retorno esperado es menor que 10**, su recompensa inmediata, porque desde A el agente es llevado a A', desde donde es probable que choque con el borde de la cuadrícula.
- El estado B tiene un valor mayor a 5, su recompensa inmediata, porque desde B el agente es llevado a B', que tiene un valor positivo.
- Desde B', la penalización (recompensa negativa) por chocar con un borde es más que compensada por la recompensa esperada de posiblemente llegar a A o B.



3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

¿PARA QUE SIRVE SABER EL $V(S)$?

- ▶ Sirve para tomar decisiones.
- ▶ Si se tienen dos posibles acciones que llevan a dos estados diferentes, $s1$ y $s2$, y se sabe que $V(s1) > V(s2)$, entonces el estado $s1$ es más prometedor a largo plazo.
- ▶ Por lo tanto, un agente debería preferir la acción que te lleva a $s1$. $V(s)$ permite al agente pensar a futuro, no solo en la recompensa inmediata.



3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

EVALUACIÓN DE POLÍTICA

EVALUACIÓN DE POLÍTICA

► El **valor de cada estado** de un MDP se puede calcular de varias formas:

✓ Sistema de N ecuaciones con N incógnitas.

$$v_{\pi}(A) = \frac{1}{4} [5 + 0.7v_{\pi}(B)] + \frac{1}{4} [0 + 0.7v_{\pi}(C)] + \frac{1}{2} [0 + 0.7v_{\pi}(A)]$$

$$v_{\pi}(B) = \frac{1}{4} [0 + 0.7v_{\pi}(A)] + \frac{1}{4} [0 + 0.7v_{\pi}(D)] + \frac{1}{2} [5 + 0.7v_{\pi}(B)]$$

✓ Algoritmo iterativo de Evaluación de Política.

✓ Métodos de Monte Carlo.

3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

EVALUACIÓN DE POLÍTICA

- ▶ ¿Qué diferencia existe entre los 2 primeros métodos?
- ▶ Sistema de ecuaciones
 - ✓ Exacto
 - ✓ Útil con MDP pequeños con pocos estados
- ▶ Evaluación de Política
 - ✓ Aproximado
 - ✓ Útil en MDP grandes o cuando no se puede usar matrices

EVALUACIÓN DE POLÍTICA

- ▶ En el algoritmo de **Evaluación de Política** se realiza lo siguiente:
 - ✓ Se calcula $v_{\pi}(s)$ para una política π fija, para todo s .
 - ✓ Se repite el cálculo hasta que $v_{\pi}(s)$ converge.
- ▶ Pseudocódigo del algoritmo de **Evaluación de Política Iterativa**:

```

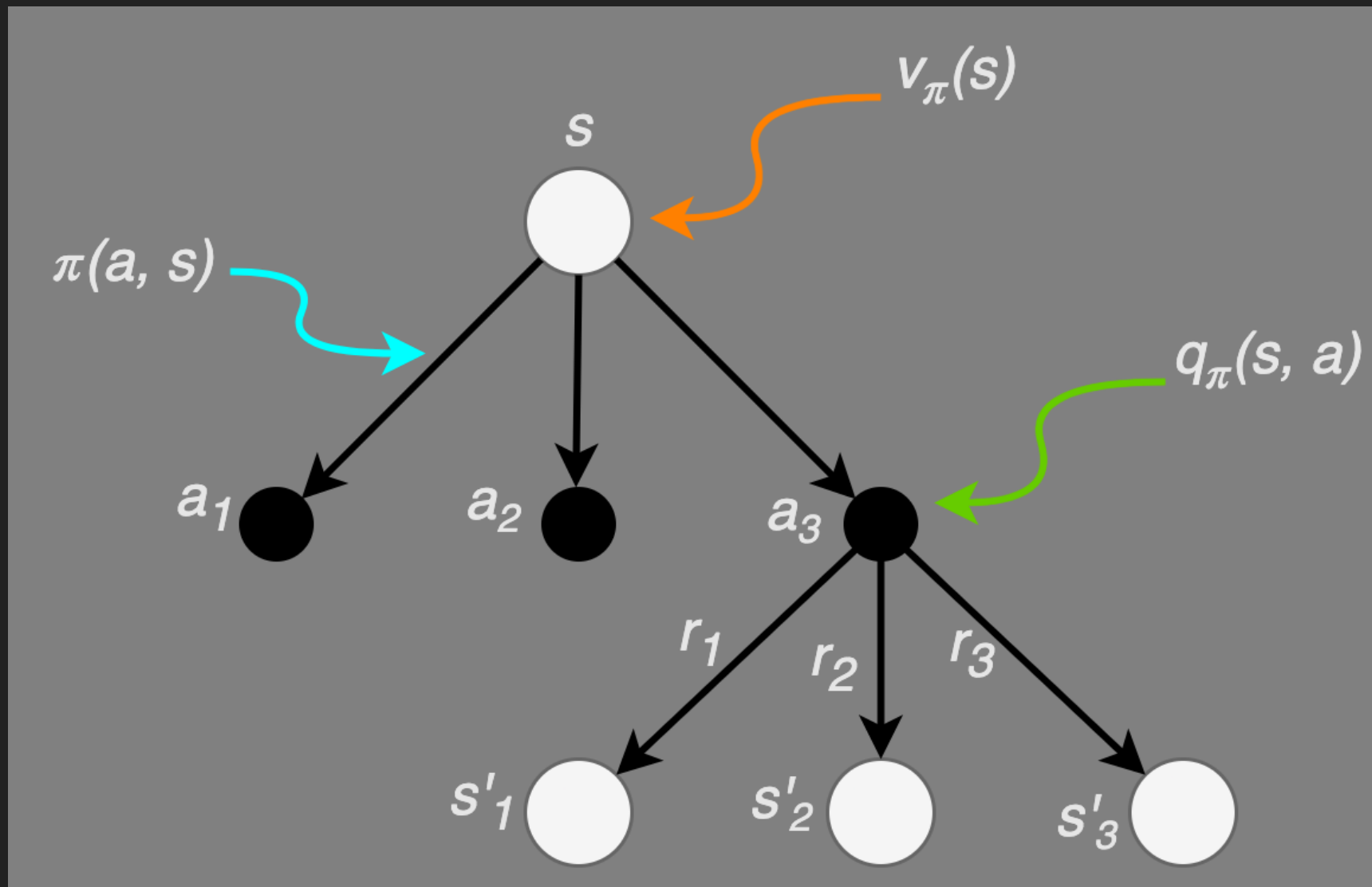
Entrada  $\pi$  (la política a ser evaluada)
Inicializar un umbral  $\theta \geq 0$  (determina la precisión de la estimación)
 $V(s) \leftarrow 0$ 

Repetir:
     $\Delta \leftarrow 0$ 
    Para cada  $s \in S$ 
         $v \leftarrow V(s)$ 
        
$$V(s) \leftarrow \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$

         $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
    hasta que  $\Delta < \theta$ 
    
```

FUNCIONES DE VALOR

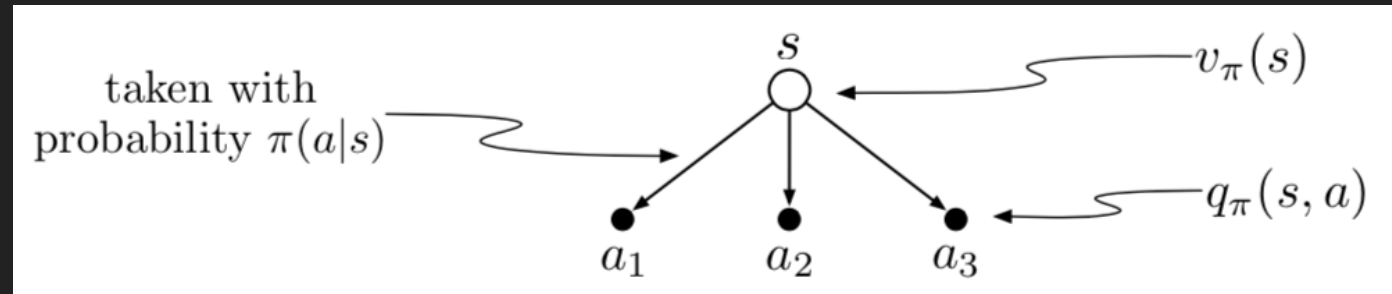
FUNCIONES DE VALOR



FUNCIONES DE VALOR

- ▶ $V(s)$ mide el valor de estar en un estado.
- ▶ El valor de un estado depende de los valores de las acciones posibles en ese estado y de la probabilidad de que se tome cada acción bajo la política actual.

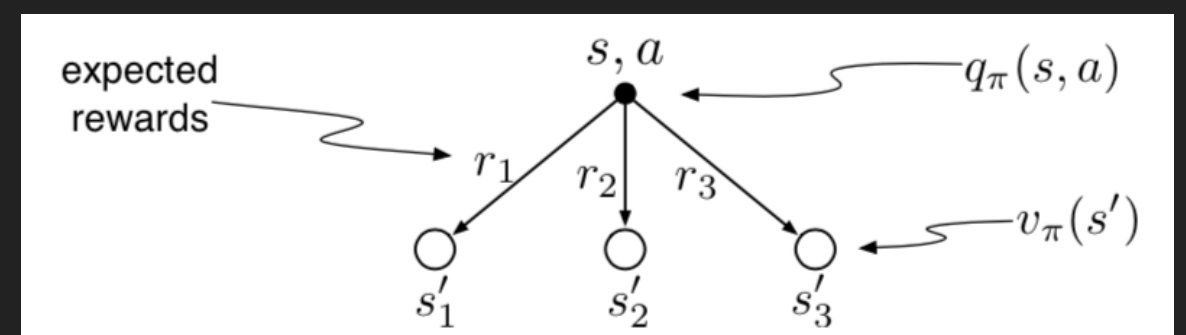
$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_{\pi}(s')],$$



- ▶ $Q(s, a)$ mide el valor de tomar una acción específica en un estado determinado y bajo una política.
- ▶ El valor de una acción depende de la próxima recompensa esperada y la suma esperada de las recompensas restantes.

~~$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_{\pi}(s')],$$~~

$$q_{\pi}(s, a) = \sum_{s',r} p(s',r|s,a) [r + \gamma v_{\pi}(s')]$$



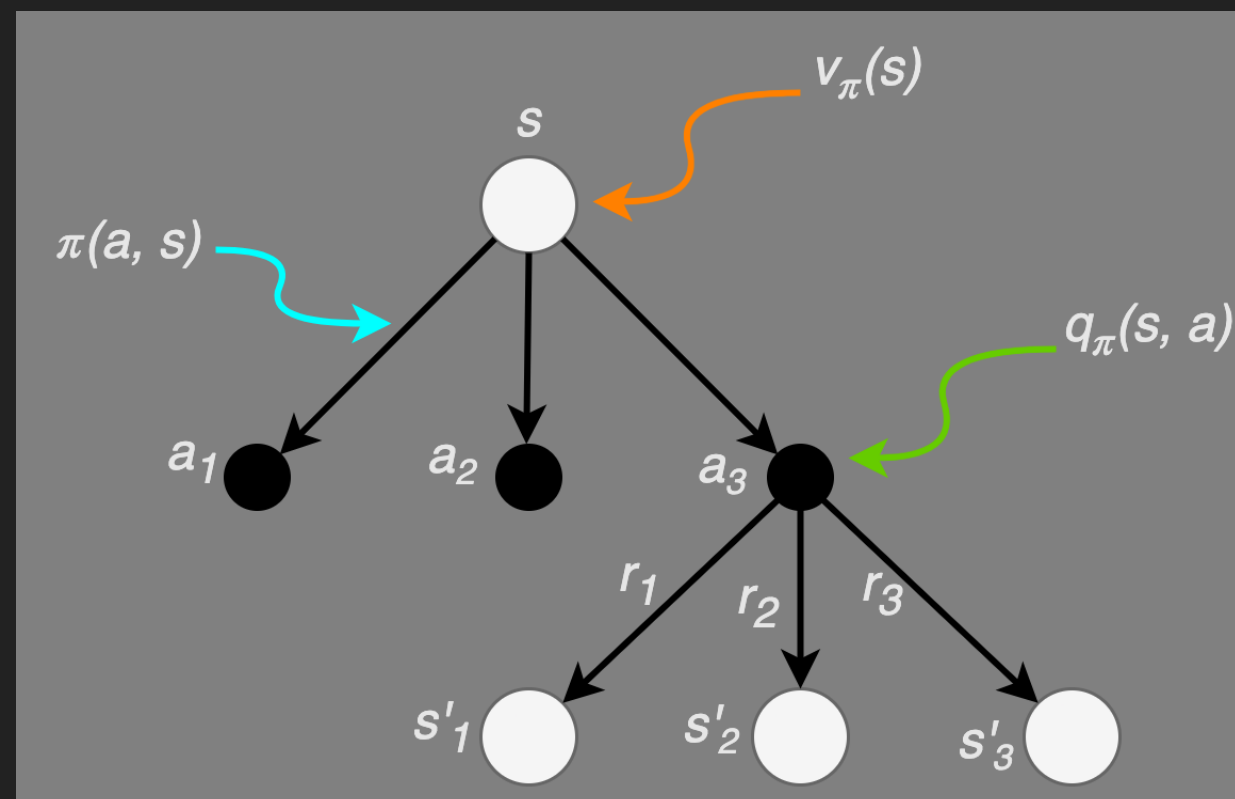
FUNCIONES DE VALOR

- Tanto $V(s)$ como $Q(s, a)$ se conocen como **Funciones de Valor**.

Funciones de Valor {

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')],$$

$$q_{\pi}(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$



ECUACIÓN DE OPTIMALIDAD DE BELLMAN

ECUACIÓN DE OPTIMALIDAD DE BELLMAN



Fuente imagen: Matrix (1999)

- **Analogía:** La ecuación de optimalidad de Bellman equivale a contar con un GPS que, para cualquier punto de una ciudad (s), no solo dice cuál es la mejor ruta, sino que otorga una puntuación de "qué tan bueno" es estar en ese punto ($V^*(s)$).
- La puntuación se basa en lo rápido que se puede llegar a un destino (la recompensa mas alta) desde ahí.

ECUACIÓN DE OPTIMALIDAD DE BELLMAN

- ▶ Resolver una tarea de aprendizaje por refuerzo significa, a grandes rasgos, **encontrar una política que genere una gran recompensa** a largo plazo.
- ▶ Una política π' se define como mejor o igual que una política π si su rendimiento esperado es mayor o igual que el de π para todos los estados.

$$\pi' \geq \pi \text{ si y solo si } v_{\pi'}(s) \geq v_{\pi}(s)$$

- ▶ Si llamamos $V(s)$ al valor de un estado, podemos llamar $V^*(s)$ al valor óptimo de un estado.
- ▶ Dado que v^* para que sea optima necesita encontrar esa política que obtenga la máxima recompensa entonces π es la incógnita y por lo tanto v^* puede expresarse sin referencia a ninguna política específica.

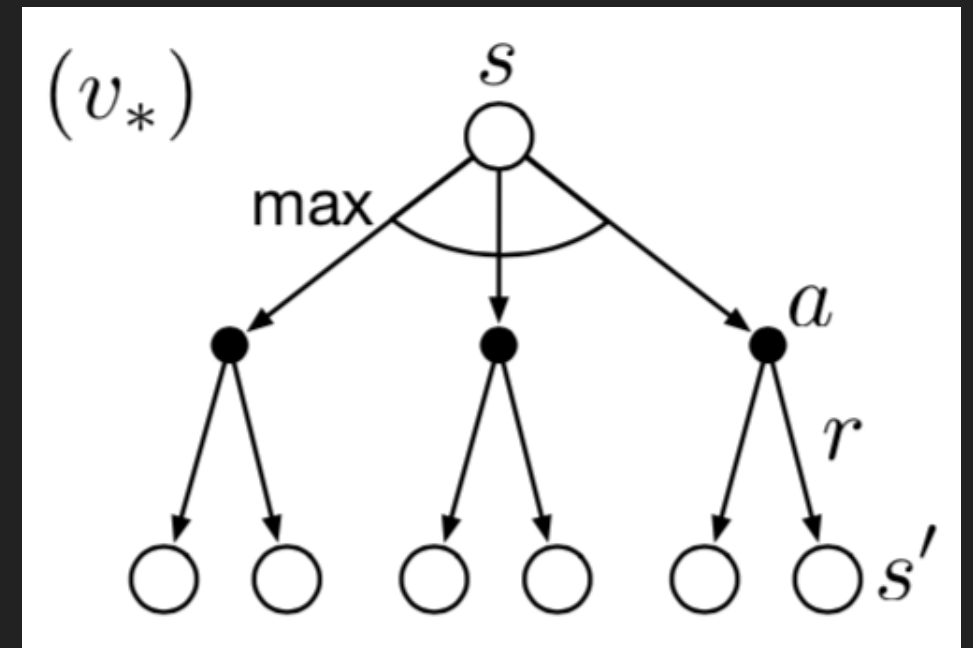
$$v_*(s) \doteq \max_{\pi} v_{\pi}(s)$$

$$\begin{aligned} v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s, a) \\ &= \max_a \mathbb{E}_{\pi_*}[G_t \mid S_t = s, A_t = a] \\ &= \max_a \mathbb{E}_{\pi_*}[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] \\ &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')]. \end{aligned}$$

ECUACIÓN DE OPTIMALIDAD DE BELLMAN

- La ecuación de optimalidad de Bellman expresa que el **valor de un estado** bajo una **política óptima** debe ser igual al rendimiento esperado de la **mejor acción** para ese estado.

$$\begin{aligned}
 v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s, a) \\
 &= \max_a \mathbb{E}_{\pi_*}[G_t \mid S_t = s, A_t = a] \\
 &= \max_a \mathbb{E}_{\pi_*}[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] \\
 &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\
 &= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')].
 \end{aligned}$$

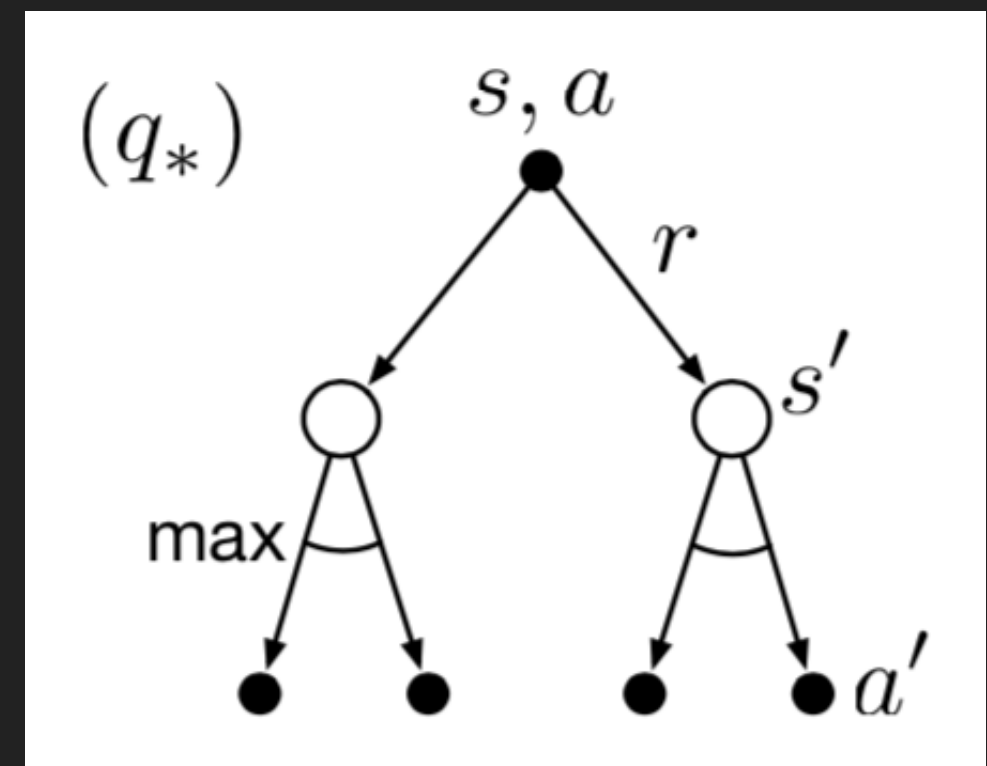


ECUACIÓN DE OPTIMALIDAD DE BELLMAN

- La ecuación de optimalidad de Bellman para q^* es:

$$q_*(s, a) \doteq \max_{\pi} q_{\pi}(s, a)$$

$$\begin{aligned} q_*(s, a) &= \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a \right] \\ &= \sum_{s', r} p(s', r | s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right]. \end{aligned}$$



MEJORA DE LA POLÍTICA

MEJORA DE LA POLÍTICA

- ▶ Vamos a suponer nuevamente el caso del **robot repositor**:
- ▶ **Estados**: **alto** (estante lleno), **medio** (estante parcialmente vacío), **bajo** (estante casi vacío)
- ▶ **Acciones**: reponer, no_reponer
- ▶ **Política**:
 - ✓ Si el estante está **alto**, el robot no repondrá ($\pi(\text{no_reponer} \mid \text{alto}) = 1$).
 - ✓ Si el estante está **medio**, el robot repondrá ($\pi(\text{reponer} \mid \text{medio}) = 1$).
 - ✓ Si el estante está **bajo**, el robot repondrá ($\pi(\text{reponer} \mid \text{bajo}) = 1$).
- ▶ después de evaluar esta política obtenemos:
 - ✓ $v_{\pi}(\text{alto}) = 6$
 - ✓ $v_{\pi}(\text{medio}) = 4$
 - ✓ $v_{\pi}(\text{bajo}) = 2$
- ▶ Estos valores nos dicen **qué tan "buenos" son cada uno de estos estados bajo la política actual (π)**.



MEJORA DE LA POLÍTICA

- ▶ ¿Se puede mejorar la política?
- ▶ La respuesta es **SI**.
- ▶ Se puede analizar el valor de las acciones en cada estado $q(s, a)$ y elegir **acciones diferentes** a la que dicta la política actual.



MEJORA DE LA POLÍTICA

- ▶ Analicemos el estado "alto": Actualmente $\pi(\text{no_reponer} \mid \text{alto}) = 1$, cambiaremos a $\pi(\text{reponer} \mid \text{alto}) = 1$.
- ▶ Supongamos que estando en estado "alto" elegimos **reponer**:
 - ✓ Hay un 80% de prob. de que el estante siga "alto" (con recompensa 1)
 - ✓ Hay un 20% de que baje a "medio" (con recompensa 0).
 - ✓ Factor de descuento γ es 0.9.
- ▶ $q_{\pi}(\text{alto}, \text{reponer}) = 0.8 * (1 + 0.9 * 6) + 0.2 * (0 + 0.9 * 4) = 5.76 + 0.72 = 6.48$
- ▶ Dado que $q_{\pi}(\text{alto}, \text{reponer}) = 6.48 > v_{\pi}(\text{alto}) = 6$, $\Rightarrow$ hay una mejora. Es mejor reponer en el estado alto que seguir la política actual de no_reponer.



MEJORA DE LA POLÍTICA

- ▶ Analicemos el estado "medio": Actualmente $\pi(\text{reponer} \mid \text{medio}) = 1$, cambiaremos a $\pi(\text{no_reponer} \mid \text{medio}) = 1$.
- ▶ Supongamos que estando en estado "medio" elegimos no_reponer:
 - ✓ Hay un 70% de prob. de que el estante pase a "bajo" (con recompensa 0)
 - ✓ Hay un 30% de que permanezca en "medio" (con recompensa 1).
 - ✓ Factor de descuento γ es 0.9.
- ▶ $q_{\pi}(\text{medio}, \text{no_reponer}) = 0.7 * (0 + 0.9 * 2) + 0.3 * (1 + 0.9 * 4) = 1.26 + 1.38 = 2.64$
- ▶ Dado que $q_{\pi}(\text{medio}, \text{no_reponer}) = 2.64 < v_{\pi}(\text{medio}) = 4$, $\Rightarrow$ seguir con la política actual (reponer) es mejor.



MEJORA DE LA POLÍTICA

► Nuestra nueva política (π') es:

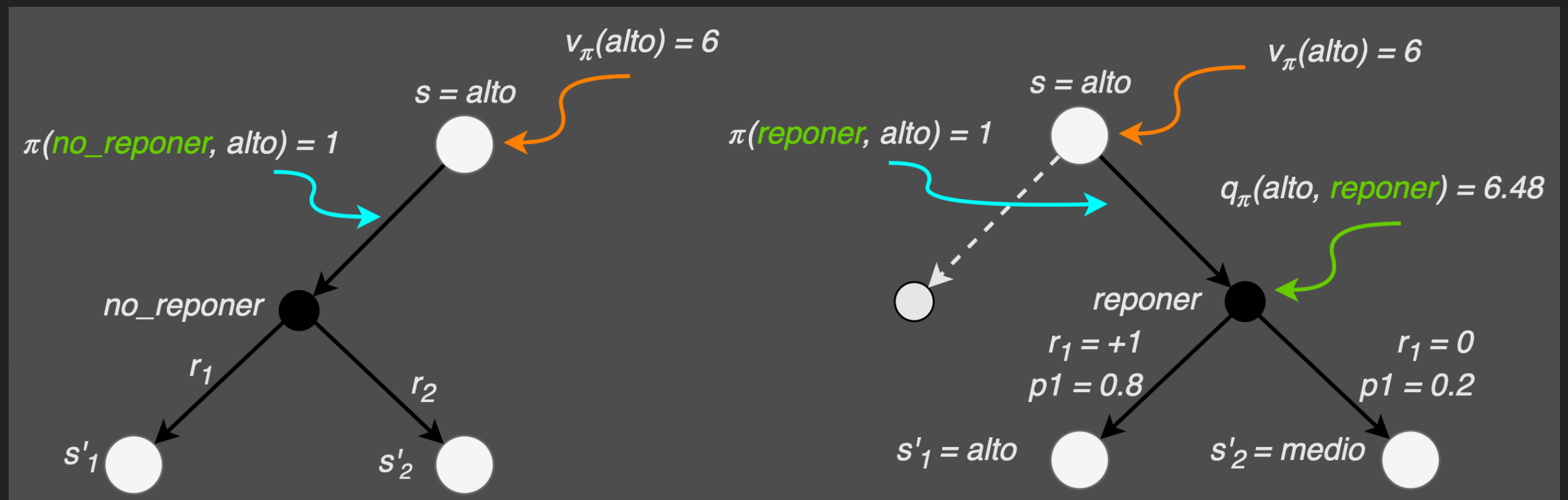
- ✓ Si el estante está alto, el robot repondrá ($\pi'(\text{reponer} \mid \text{alto}) = 1$).
- ✓ Si el estante está medio, el robot repondrá ($\pi'(\text{reponer} \mid \text{medio}) = 1$).
- ✓ Si el estante está bajo, el robot repondrá ($\pi'(\text{reponer} \mid \text{bajo}) = 1$).

- Habría que iterar y volver a evaluar esta nueva política π' (calculando $v_{\pi'}(s)$) y luego intentar mejorarla nuevamente.
- Este proceso de evaluación y mejora puede repetirse hasta que no podamos encontrar más mejoras, en cuyo caso habremos encontrado la política óptima.



MEJORA DE LA POLÍTICA

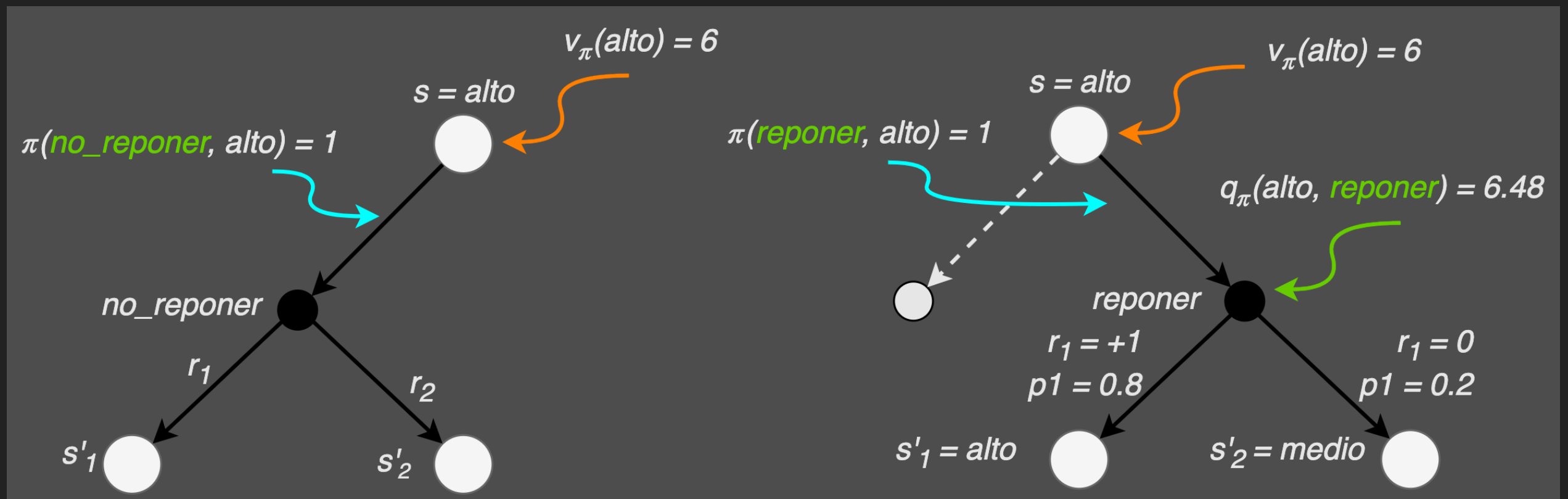
- Formalmente el objetivo de calcular la función de valor $v(s)$ o $q(s, a)$ de una política es tratar de encontrar mejores políticas.
- ¿Sería mejor o peor cambiar a la nueva política? Una forma de saberlo es considerar seleccionar a en s y, a partir de entonces, seguir la política existente.



- Si $q_\pi(s, a) > v_\pi(s)$, entonces tomar la acción a en el estado s y luego seguir la política π es mejor que simplemente seguir la política π desde el estado s .

MEJORA DE LA POLÍTICA

- Podemos traducir el usar $v_\pi(s)$ como: "Si sigo haciendo lo que mi política actual me dice que haga, esto es lo que puedo esperar obtener a largo plazo $v_\pi(s) = 6$ ".
- Podemos traducir el usar $q_\pi(s, a)$ como: "Si hago algo **diferente** a lo que mi política actual me dice que haga **solo una vez**, y luego sigo haciendo lo que mi política actual me dice que haga, esto es lo que puedo esperar obtener a largo plazo $q_\pi(s, a) = 6.48$ ".



TEOREMA DE MEJORA DE LA POLÍTICA

TEOREMA DE MEJORA DE POLÍTICAS

- ▶ Si tengo una política π , y encuentro otra política π' que en cada estado elige una acción que parece mejor (o al menos igual de buena) según los valores de π , entonces π' es una política igual o mejor que π .

TEOREMA DE MEJORA DE POLÍTICAS

- ▶ Si $q_{\pi}(s, a) > v_{\pi}(s)$ entonces cabría esperar que fuera aún mejor seleccionar a (reponer) cada vez que se encuentra s (alto), y que la nueva política fuera mejor en general.
- ▶ Que esto sea cierto es un caso especial de un resultado general llamado **Teorema de mejora de políticas**.
- ▶ Sea π y π' dos políticas deterministas. Si $q_{\pi}(s, \pi'(s)) \geq v_{\pi}(s)$ para todos los estados s , entonces la política π' es al menos tan buena como la política π (es decir, $v_{\pi'}(s) \geq v_{\pi}(s)$ para todos los estados s).
- ▶ Si existe una desigualdad estricta ($>$) para al menos un estado, entonces la política π' es estrictamente mejor que la política π .
- ▶ En otras palabras: Si para todos los estados s , el valor de tomar la acción que π' recomienda (y luego seguir π) es al menos tan bueno como seguir π directamente, entonces π' es una mejora sobre π .

TEOREMA DE MEJORA DE POLÍTICAS

- Una vez que encontré una política π' que es mejor que la anterior política π , el siguiente paso es considerar **cambiar la política en todos los estados y para todas las acciones**. Esto lleva a la idea de una **"greedy policy"** π' :

$$\begin{aligned}\pi'(s) &= \operatorname{argmax}_a q_{\pi}(s, a) \\ &= \operatorname{argmax}_a E[R_{t+1} + \gamma^* v_{\pi}(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \operatorname{argmax}_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma^* v_{\pi}(s')]\end{aligned}$$

- La **"greedy policy"** elige, en cada estado, la acción que parece mejor **a corto plazo** (después de un paso de **"lookahead"**) según la función de valor de la política original π .
- Si la **"greedy policy"** π' es igual a la política original π , entonces la política original π es óptima. En ese caso la política actual es la mejor.
- Este proceso de evaluar una **política** (calculando v_{π}) y luego mejorarla (creando una **"greedy policy"** π') se puede repetir iterativamente para encontrar la **política óptima**.

PROGRAMACIÓN DINÁMICA

¿QUÉ ES LA PROGRAMACIÓN DINÁMICA?

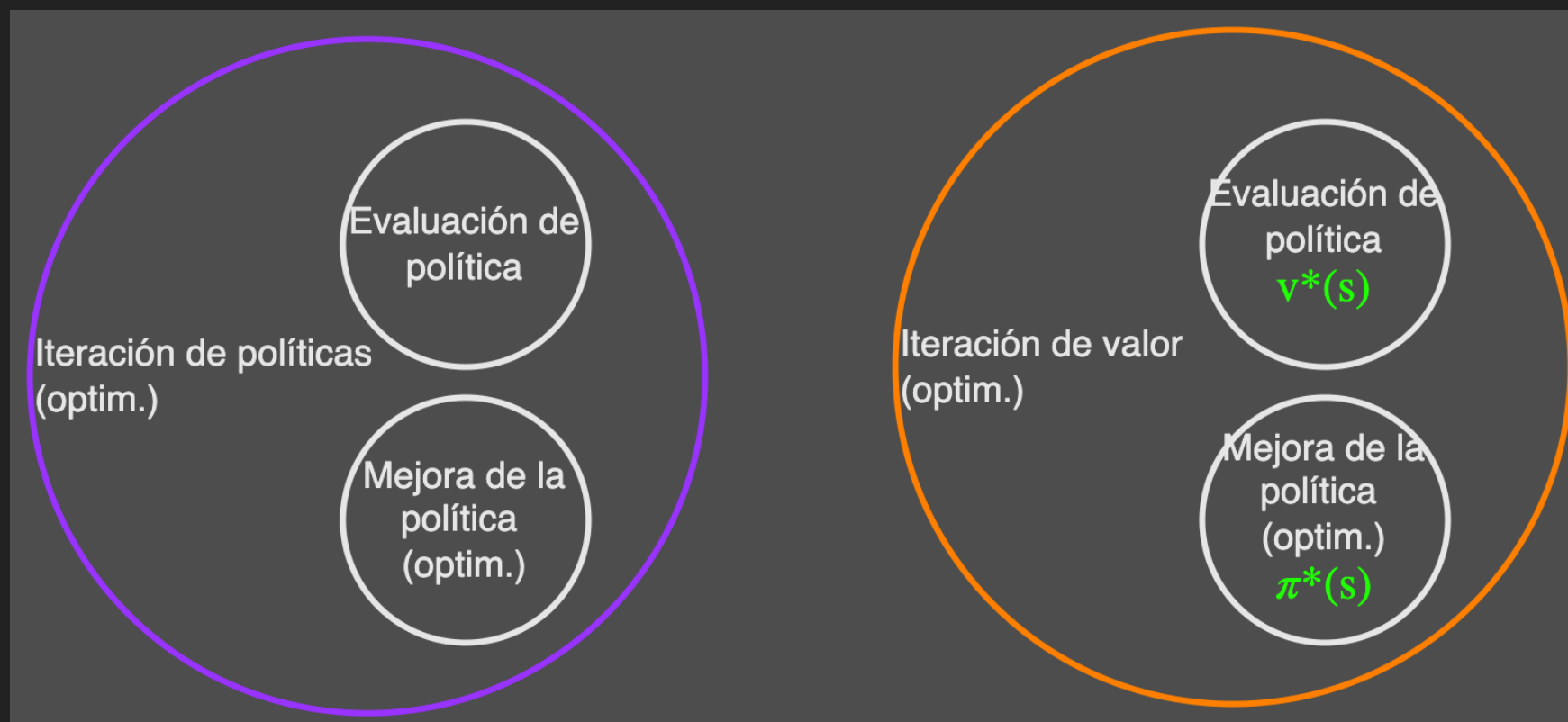
- ▶ **Programación Dinámica** (DP) en RL es un conjunto de algoritmos para resolver **Procesos de Decisión de Markov (MDP)** cuando el modelo del entorno es completamente conocido. Utiliza la Ecuación de Bellman para calcular **funciones de valor** y encontrar **políticas óptimas** mediante métodos como **Iteración de Política** e **Iteración de Valor**.
 - ▶ Requiere conocer la probabilidad de transición $p(s', r | s, a)$, lo que lo hace **model-based**.
 - ▶ Es computacionalmente costoso en problemas con muchos estados.
-
- ▶ **Programación Dinámica** (DP) **clásica** es una técnica de optimización que divide un problema en subproblemas más pequeños y solapa sus soluciones para evitar cálculos redundantes. Se usa en problemas con estructura óptima de subproblemas y superposición de soluciones.
 - ▶ Se aplica en problemas como Fibonacci, la mochila, cadenas de ADN, caminos más cortos, etc.
 - ▶ Se puede implementar de forma recursiva con **memoization** o iterativa con una tabla.

ITERACIÓN DE LA POLÍTICA E ITERACIÓN DE VALOR

- ▶ Hasta ahora vimos 2 algoritmos en cuanto a resolución de MDPs:
 - ✓ **Evaluación de Política (predicción)** $\Rightarrow$ dado π (fijo), determino $v(s)$.
 - ✓ **Mejora de la Política** $\Rightarrow$ cambio π (parcialmente) por π' , y luego comparo si $v_{\pi'}(s) \geq v_{\pi}(s)$.

ITERACIÓN DE LA POLÍTICA E ITERACIÓN DE VALOR

- ▶ ¿Cómo puedo determinar una política óptima en lugar de tener una política fija la cual no sé si es optima?
- ▶ Existen 2 formas:
 - ✓ Iteración de Políticas.
 - ✓ Iteración de Valor.



ITERACIÓN DE LA POLÍTICA E ITERACIÓN DE VALOR

► ¿En qué consiste la Iteración de Políticas y la Iteración de Valor?

Policy Iteration	Value Iteration
fijar un valor de π cualquiera Repetir evaluación de politica() mejora de la politica() hasta que π no cambie	fijar un valor de π cualquiera Repetir calcular $v^*(s)$ hasta que $v^*(s)$ no cambie Para cada estado de $v^*(s)$ calcular $\pi^*(s)$



ITERACIÓN DE LA POLÍTICA

- ▶ ¿En que consiste concretamente la **iteración de la política**?
- ▶ Luego de que mejoramos una política a partir de $v_{\pi}(s)$ y obtenemos π' , se puede determinar $v_{\pi'}(s)$ y así obtener π'' .

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \dots \xrightarrow{I} \pi_* \xrightarrow{E} v_{\pi_*}$$

- ▶ dónde **E** es la evaluación de la política y la **I** es la mejora de la política (Improvement).
- ▶ Ambos procesos (**E** e **I**) convergen hacia una política óptima y hacia una función de valor óptima.
- ▶ Este método se conoce como **iteración de la política**.

ITERACIÓN DE LA POLÍTICA

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

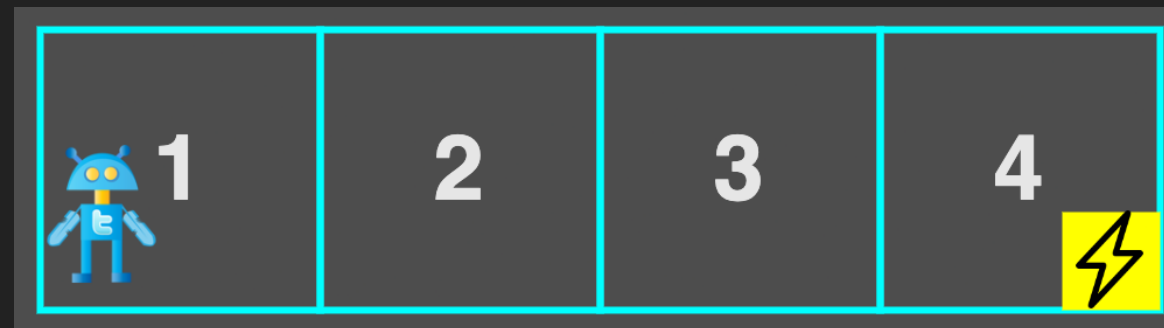
$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

ITERACIÓN DE LA POLÍTICA

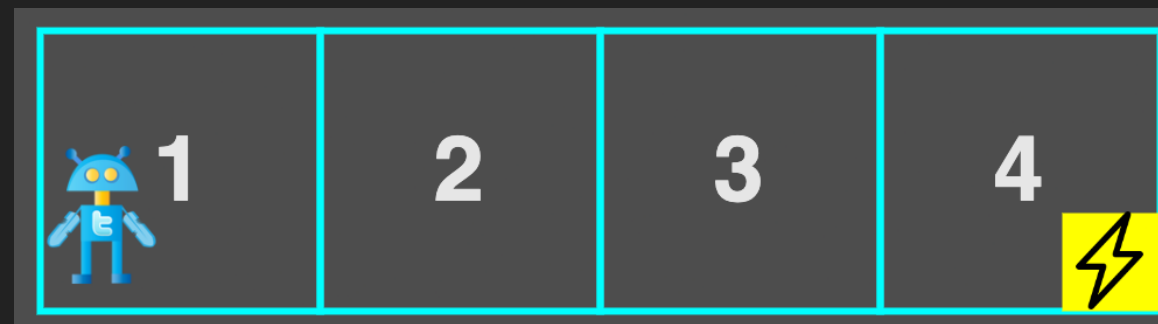
- ▶ Ejemplo: Robot limpiador (de un pasillo)



- ▶ El robot limpia un pequeño pasillo dividido en 4 casillas numeradas del 1 al 4.
- ▶ El robot puede estar en cualquier casilla.
- ▶ Su objetivo es llegar a la casilla 4, donde se encuentra la estación de carga.

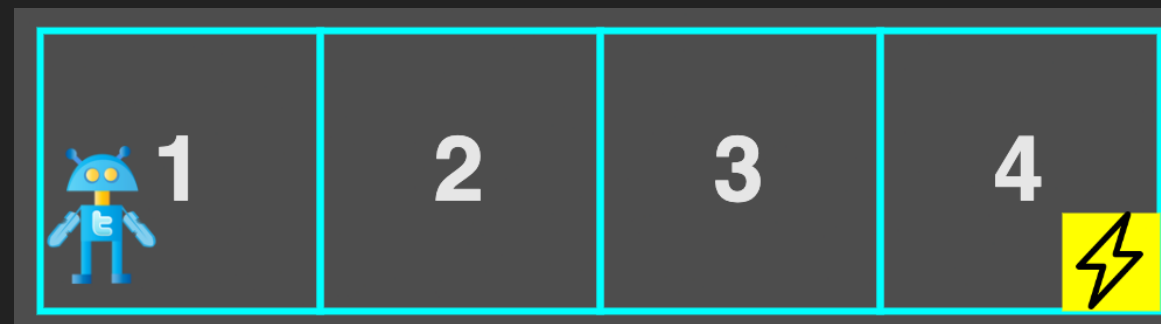
ITERACIÓN DE LA POLÍTICA

- ▶ **Estados:** Las casillas donde puede estar el robot: $\{1, 2, 3, 4\}$ (4 es el estado terminal)
- ▶ **Acciones:** El robot solo puede moverse a la izquierda o a la derecha (si es posible).
- ▶ **Recompensas:**
 - ✓ -1: Recibe penalización de -1 en cualquier casilla que no sea la estación de carga (4).
 - ✓ 0: Al llegar a la casilla 4 (estación de carga).
- ▶ **Transiciones:**
 - ✓ Si el robot intenta ir a la izquierda desde la casilla 1, permanece en la casilla 1.
 - ✓ Si el robot intenta ir a la derecha desde la casilla 4, permanece en la casilla 4.
 - ✓ En cualquier otra casilla, el movimiento es determinista (si elige a la derecha, se mueve a la derecha; si elige a la izquierda, se mueve a la izquierda).
- ▶ **Factor de Descuento (γ):** 0.9 (Damos mucho valor a las recompensas futuras)



ITERACIÓN DE LA POLÍTICA

- ▶ El proceso de **iteración de políticas** lo analizaremos en 4 pasos:
- ✓ Paso 1: Inicialización de la Política y la Función de Valor.
- ✓ Paso 2: Evaluación de la Política.
- ✓ Paso 3: Mejora de la Política.
- ✓ Paso 4: Repetir Evaluación y Mejora. ↻



ITERACIÓN DE LA POLÍTICA

► **Paso 1 de 4:** Inicialización de la Política y la Función de Valor.

► Política inicial (π): Asignamos una acción aleatoria a cada estado (excepto al estado terminal):

✓ $\pi(1) = \text{Derecha (D)}$

✓ $\pi(2) = \text{Izquierda (I)}$

✓ $\pi(3) = \text{Derecha (D)}$

✓ $\pi(4) = \text{No aplica (estado terminal)}$

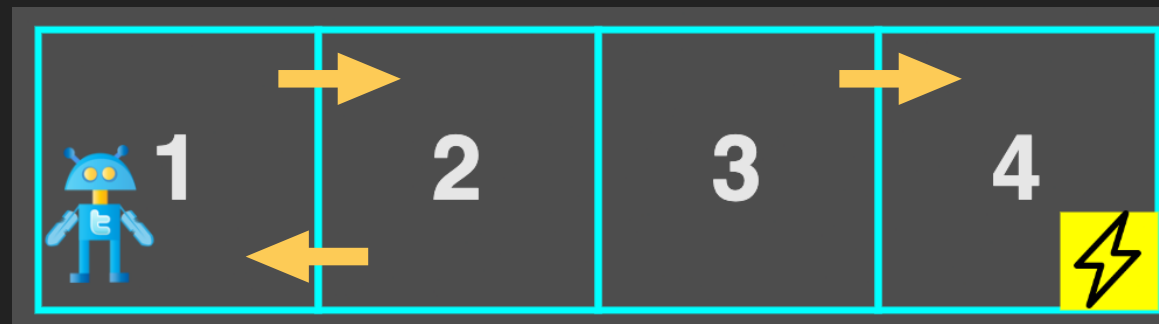
► Función de valor inicial (V): Inicializamos los valores de todos los estados en 0.

✓ $V(1) = 0$

✓ $V(2) = 0$

✓ $V(3) = 0$

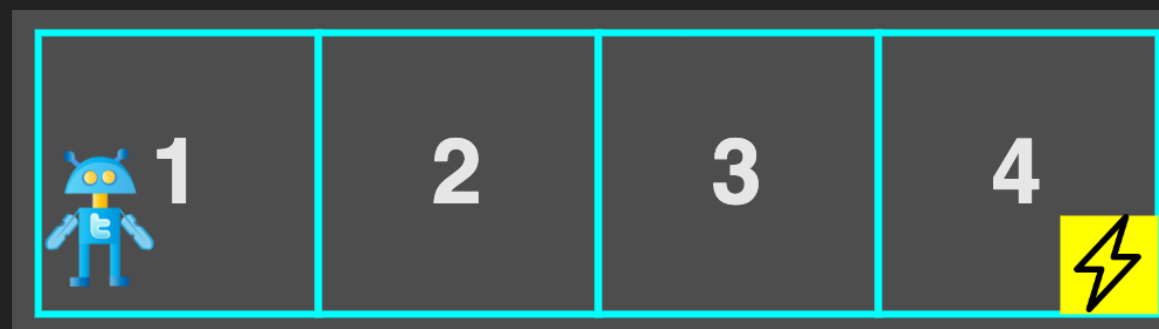
✓ $V(4) = 0$



ITERACIÓN DE LA POLÍTICA

- ▶ **Paso 2 de 4:** Evaluación de la Política.
- ▶ Calculamos la función de valor v para la política actual π .

$$V(s) = R(s, \pi(s)) + \gamma * V(s')$$



ITERACIÓN DE LA POLÍTICA

► **Paso 2 de 4:** Evaluación de la Política.

► Iteramos hasta que la **función de valor converja** (es decir, hasta que los cambios en los valores de los estados sean muy pequeños).

► Iteración 1:

✓ $V(1) = -1 + 0.9 * V(2) \Rightarrow V(1) = -1 + 0.9 * 0 = -1$

✓ $V(2) = -1 + 0.9 * V(1) \Rightarrow V(2) = -1 + 0.9 * -1 = -1.9$

✓ $V(3) = -1 + 0.9 * V(4) \Rightarrow V(3) = -1 + 0.9 * 0 = -1$

✓ $V(4) = 0$

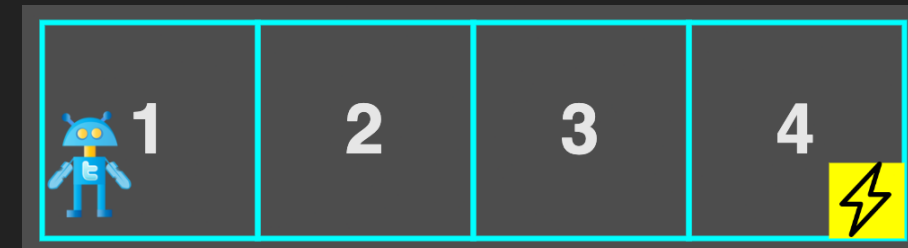
► Iteración 2:

✓ $V(1) = -1 + 0.9 * V(2) \Rightarrow V(1) = -1 + 0.9 * -1.9 = -2.71$

✓ $V(2) = -1 + 0.9 * V(1) \Rightarrow V(2) = -1 + 0.9 * -2.71 = -3.439$

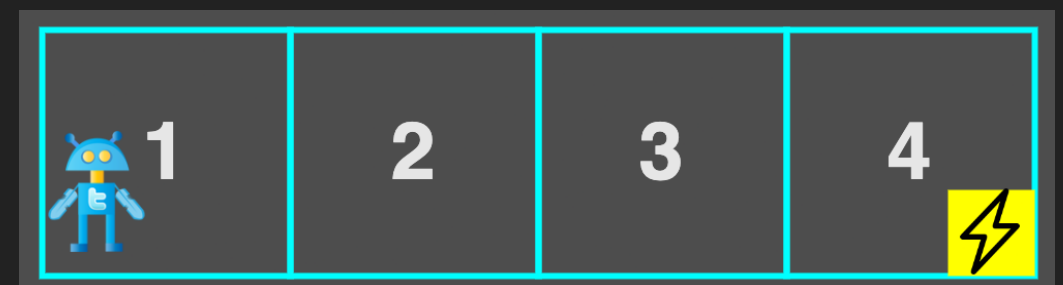
✓ $V(3) = -1 + 0.9 * V(4) \Rightarrow V(3) = -1 + 0.9 * 0 = -1$

✓ $V(4) = 0$



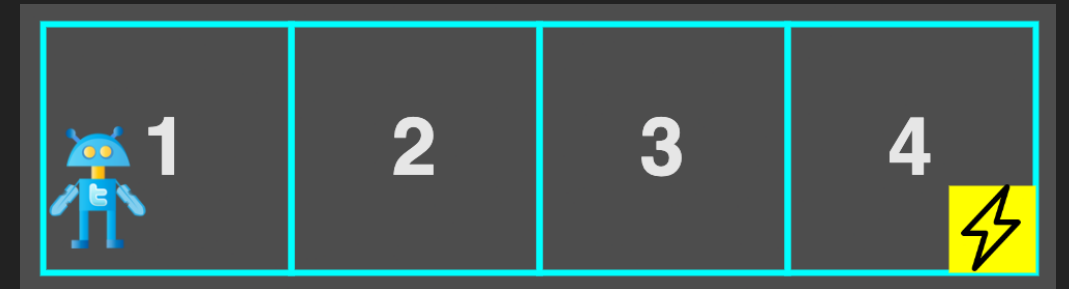
ITERACIÓN DE LA POLÍTICA

- ▶ **Paso 2 de 4:** Evaluación de la Política.
- ▶ Iteraciones siguientes... Este proceso continúa hasta que los valores de $V(1)$, $V(2)$ y $V(3)$ dejen de cambiar significativamente.
- ▶ Iteración n:
 - ✓ $V(1) = -2.71$
 - ✓ $V(2) = -1.9$
 - ✓ $V(3) = -1$
 - ✓ $V(4) = 0$



ITERACIÓN DE LA POLÍTICA

► **Paso 3 de 4:** Mejora de la Política.



- Una vez que tenemos una estimación precisa de la función de valor para la política actual, intentamos mejorar la política. Para cada estado, consideramos todas las posibles acciones y elegimos la acción que maximiza el valor esperado:

$$\pi(s) = \operatorname{argmax}_a [R(s, a) + \gamma * V(s')]$$

► donde:

- ✓ argmax_a significa "la acción a que maximiza la expresión entre corchetes"
- ✓ $R(s, a)$: Recompensa inmediata al tomar la acción a en el estado s
- ✓ $V(s')$: Valor del estado siguiente después de tomar la acción a en el estado s

ITERACIÓN DE LA POLÍTICA

► Paso 3 de 4: Mejora de la Política.

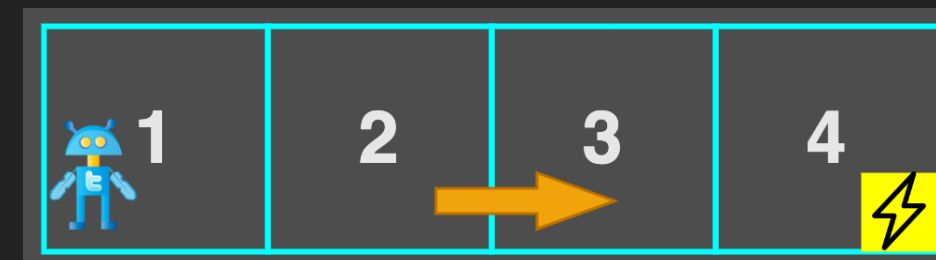
► Realizamos la mejora de la política para cada estado:

► Estado 1:

- ✓ Acción Izquierda (I): $R(1, I) + \gamma * V(1) = -1 + 0.9 * -2.71 = -3.439$
- ✓ Acción Derecha (D): $R(1, D) + \gamma * V(2) = -1 + 0.9 * -1.9 = -2.71$
- ✓ Como $-2.71 > -3.439$, $\pi(1) = \text{Derecha (I)}$ (La política recomienda ir a la derecha en el estado 1)

► Estado 2:

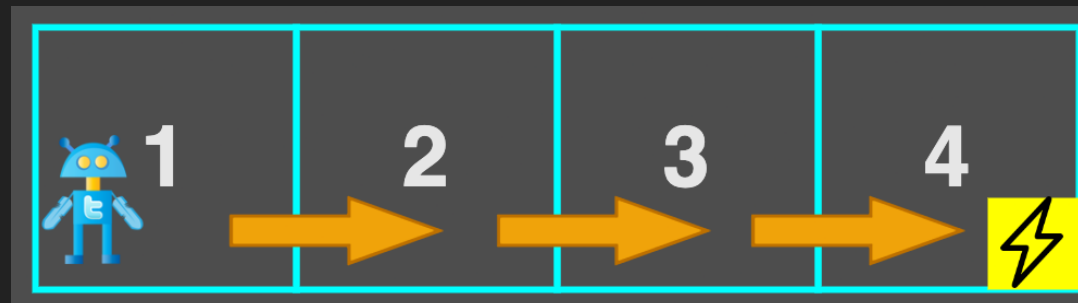
- ✓ Acción Izquierda (I): $R(2, I) + \gamma * V(1) = -1 + 0.9 * -2.71 = -3.439$
- ✓ Acción Derecha (D): $R(2, D) + \gamma * V(3) = -1 + 0.9 * -1 = -1.9$
- ✓ Como $-1.9 > -3.439$, $\pi(2) = \text{Derecha (D)}$ (La política recomienda ir a la derecha en el estado 2)



ITERACIÓN DE LA POLÍTICA

- ▶ **Paso 4 de 4:** Repetir Evaluación y Mejora.
- ▶ Volvemos al **paso 2** (Evaluación de la Política) y repetimos el proceso con la nueva política. Calculamos una nueva función de valor para esta política mejorada y luego intentamos mejorar la política nuevamente.
- ▶ Continuar **hasta que la política ya no cambie**. Cuando la política se mantiene igual después de una iteración de mejora, significa que hemos encontrado la política óptima.
- ▶ Política Óptima (Después de 4 iteraciones):

- ✓ $\pi(1) = \text{Derecha (D)}$
- ✓ $\pi(2) = \text{Derecha (D)}$
- ✓ $\pi(3) = \text{Derecha (D)}$
- ✓ $\pi(4) = \text{No aplica}$



- ▶ Es decir, la **política óptima** es que el robot siempre se mueva a la derecha hasta llegar a la estación de carga.

ITERACIÓN DE VALOR

ITERACIÓN DE VALOR

- ▶ ¿En que consiste concretamente la **iteración de valor**?
- ▶ En la **Iteración de Políticas**, debemos iterar hasta que la función de valor $v(s)$ converja (o esté "suficientemente cerca" de la convergencia) para la política actual antes de **mejorar la política**.
- ▶ Este proceso de **evaluación** puede ser costoso computacionalmente, especialmente para problemas grandes.

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

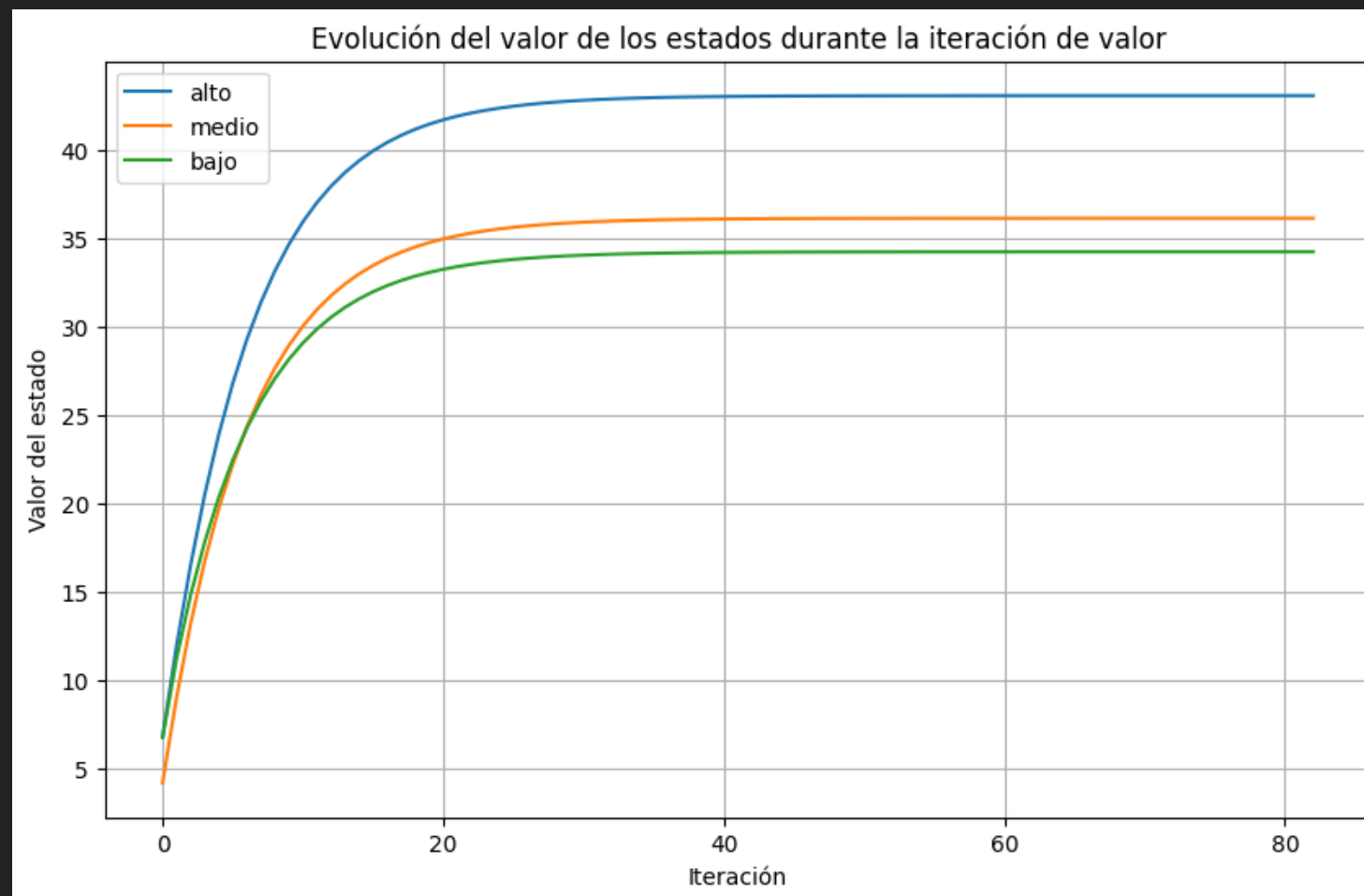
$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

ITERACIÓN DE VALOR

- ▶ En la práctica, después de un cierto número de iteraciones, la **evaluación de la política** puede tener **poco efecto sobre la convergencia final** y también sobre la **determinación de la política óptima**.
- ▶ Esto significa que podemos **truncar el proceso de evaluación de la política** sin sacrificar significativamente la calidad de la solución.



ITERACIÓN DE VALOR

- ▶ Una forma de **truncar** el proceso de **evaluación de la política** es **calcular la mejor acción posible en cada estado** y usar esa acción para actualizar la función de valor $v(s)$.
- ▶ Esto es precisamente lo que hace la **Iteración de Valor**.

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

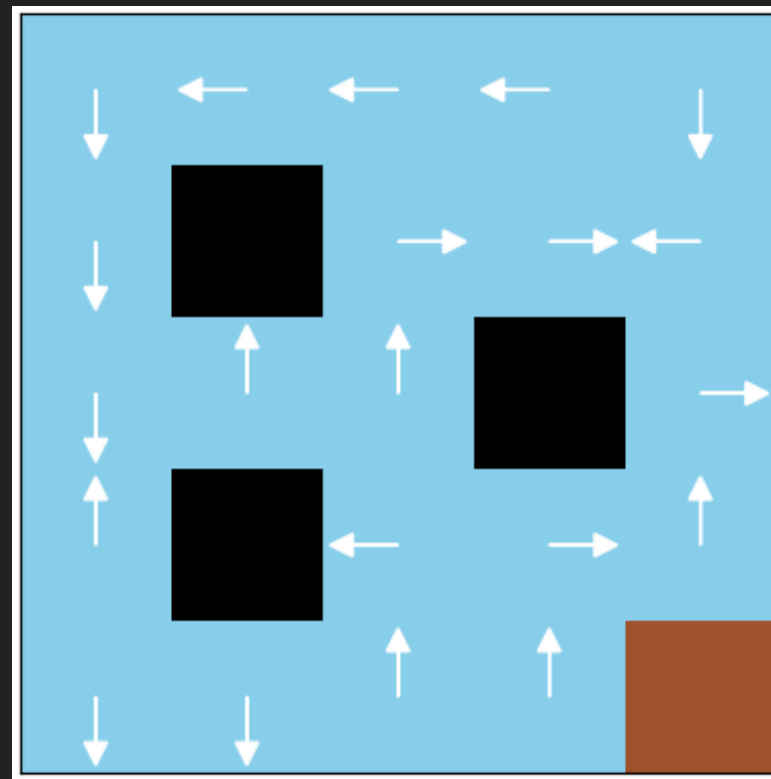
```
|  $\Delta \leftarrow 0$   
| Loop for each  $s \in \mathcal{S}$ :  
|    $v \leftarrow V(s)$   
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$   
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$   
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \arg \max_a \sum_{s',r} p(s', r | s, a) [r + \gamma V(s')]$$

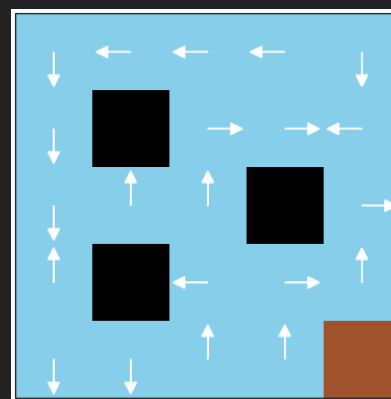
ITERACIÓN DE VALOR

- ▶ Ejemplo: Navegación de un Barco en un Océano
- ▶ El océano está dividido en una cuadrícula de celdas.
- ▶ Algunas celdas son agua navegable, otras son rocas o islotes (obstáculos), y una celda es el muelle (destino final).
- ▶ El **objetivo** del barco es **llegar al muelle lo más rápido posible evitando las rocas**.



ITERACIÓN DE VALOR

- ▶ **Estados:** Se representan por cada **celda navegable** en la cuadrícula del océano. El muelle es el **estado terminal**.
- ▶ **Acciones:** El barco puede moverse en cuatro direcciones: **Norte, Sur, Este, Oeste**. Si el barco intenta moverse hacia una roca o fuera de los límites del océano, permanece en su celda actual.
- ▶ **Recompensas:**
 - ✓ **-1:** Por cada movimiento en una celda navegable que no sea el muelle.
 - ✓ **0:** Al llegar al muelle.
- ▶ **Factor de Descuento (γ):** **0.9** (se busca llegar al muelle rápidamente).



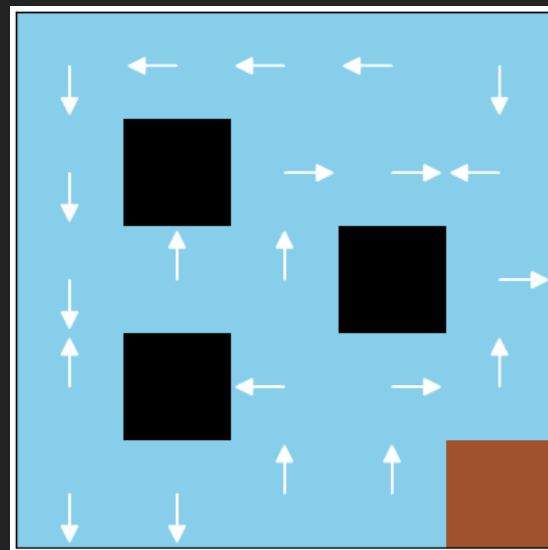
ITERACIÓN DE VALOR

► El proceso de **iteración de valor** lo analizaremos en 3 pasos:

✓ Paso 1: Inicialización.

✓ Paso 2: Evaluación de la Política.

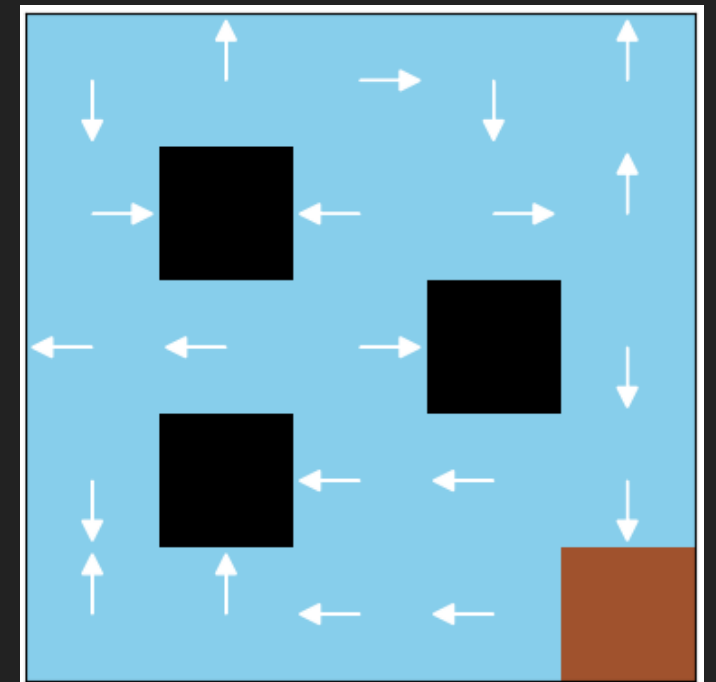
✓ Paso 3: Extracción de la Política Óptima



ITERACIÓN DE VALOR

- ▶ **Paso 1 de 3:** Inicialización.
- ▶ **Entorno:** Océano de 5x5, muelle en (4, 4), rocas en (1, 1), (2, 3), (3, 1).
- ▶ Factor de Descuento (γ): 0.9
- ▶ Función de Valor Inicial ($V(s)$): 0 para todas las celdas.
- ▶ **Estado** Inicial: (0, 0)

```
# Definición del Entorno
LADO_OCEANO = 5 # Tamaño del oceano (cuadrado de 5x5)
MUELLE = (4, 4) # Coordenadas del muelle (fila, columna)
ROCAS = [(1, 1), (2, 3), (3, 1)] # Coordenadas de las rocas
ACCIONES = ['Norte', 'Sur', 'Este', 'Oeste']
RECOMPENSA_MOVIMIENTO = -1
RECOMPENSA_MUELLE = 0
RECOMPENSA_ROCA = -100
FACTOR_DESCUENTO = 0.9
TOLERANCIA = 1e-6
```



ITERACIÓN DE VALOR

► Paso 2 de 3: Iteración de Valor.

► Iteración 1:

► 1. Estado (0, 0):

✓ Acciones Posibles: Norte, Sur, Este, Oeste.

► 2. Calculamos el valor de cada acción:

✓ Norte: $R(0, 0, \text{Norte}) + \gamma * V(1, 0) = -1 + 0.9 * 0 = -1$

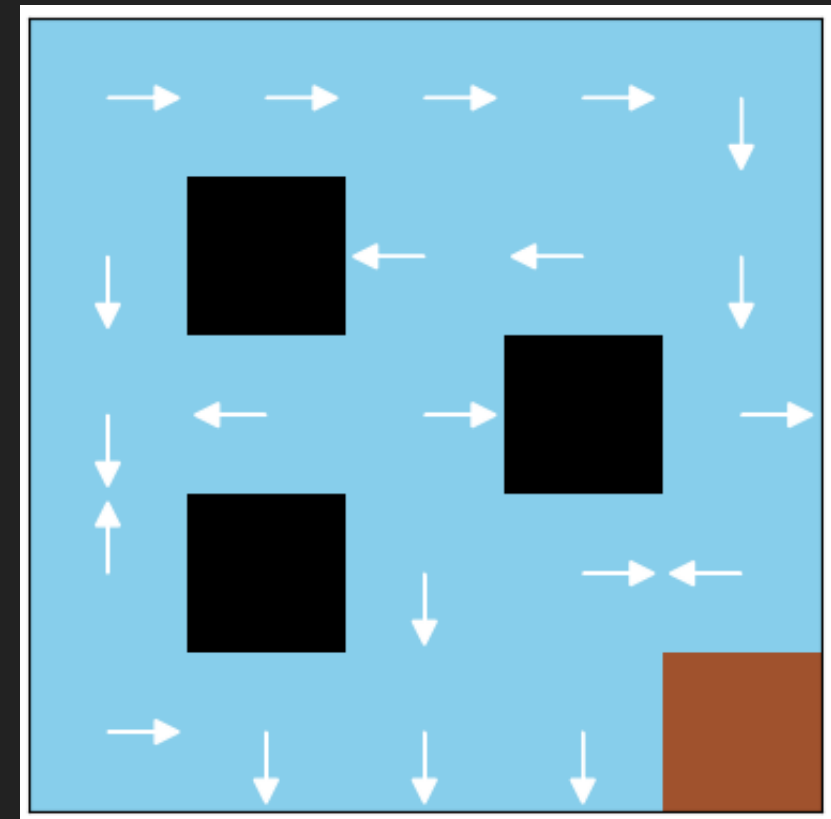
✓ Sur: $R(0, 0, \text{Sur}) + \gamma * V(0, 1) = -1 + 0.9 * 0 = -1$

✓ Este: $R(0, 0, \text{Este}) + \gamma * V(0, 1) = -1 + 0.9 * 0 = -1$

✓ Oeste: $R(0, 0, \text{Oeste}) + \gamma * V(0, 0) = -1 + 0.9 * 0 = -1$

► $V(0, 0) = \max(-1, -1, -1, -1) = -1$

En este punto, cualquier acción es igualmente buena (o igualmente mala). Arbitrariamente, elegimos Este. El barco se mueve a (0, 1).



ITERACIÓN DE VALOR

► **Paso 2 de 3:** Evaluación de la Política.

► Iteración 2:

► 1. Estado $(0, 1)$:

✓ Acciones Posibles: Norte, Sur, Este, Oeste.

► 2. Calculamos el valor de cada acción (usando el valor actualizado de $V(0, 0) = -1$):

✓ Norte: $R(0, 1, \text{Norte}) + \gamma * V(1, 1) = -1 + 0.9 * 0 = -1$

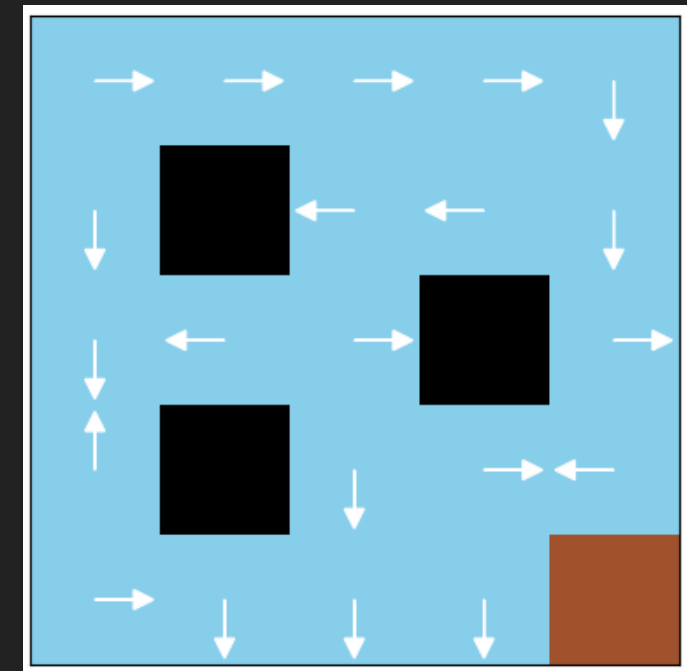
✓ Sur: $R(0, 1, \text{Sur}) + \gamma * V(0, 0) = -1 + 0.9 * (-1) = -1.9$

✓ Este: $R(0, 1, \text{Este}) + \gamma * V(0, 2) = -1 + 0.9 * 0 = -1$

✓ Oeste: $R(0, 1, \text{Oeste}) + \gamma * V(0, 0) = -1 + 0.9 * (-1) = -1.9$

► $V(0, 1) = \max(-1, -1.9, -1, -1.9) = -1$

En este punto, Norte, Este y Oeste son igualmente buenas. Arbitrariamente, elegimos Este. El barco se mueve a $(0, 2)$.



ITERACIÓN DE VALOR

► Paso 2 de 3: Evaluación de la Política.

► Iteración 3:

► 1. Estado (0, 2):

✓ Acciones Posibles: Norte, Sur, Este, Oeste.

► 2. Calculamos el valor de cada acción (usando el valor actualizado de $V(0, 0) = -1$ y $V(0, 1) = -1$):

✓ Norte: $R(0, 2, \text{Norte}) + \gamma * V(1, 2) = -1 + 0.9 * 0 = -1$

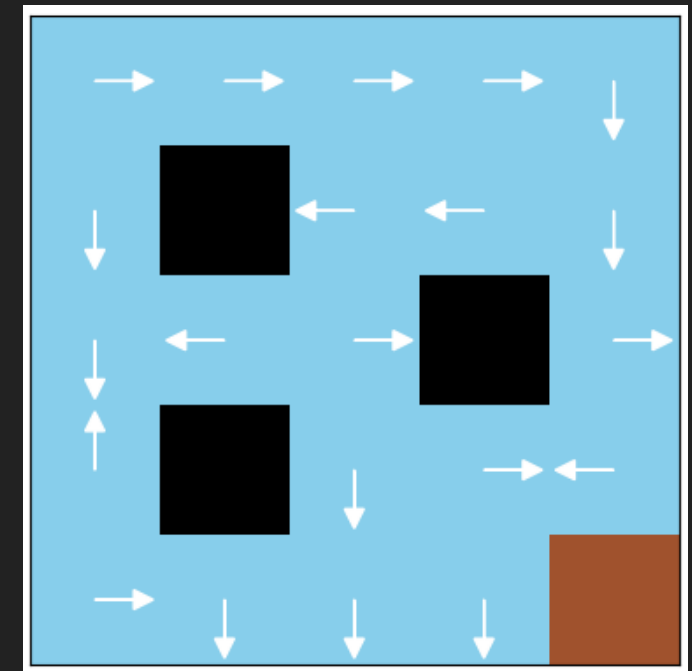
✓ Sur: $R(0, 2, \text{Sur}) + \gamma * V(0, 1) = -1 + 0.9 * (-1) = -1.9$

✓ Este: $R(0, 2, \text{Este}) + \gamma * V(0, 3) = -1 + 0.9 * 0 = -1$

✓ Oeste: $R(0, 2, \text{Oeste}) + \gamma * V(0, 1) = -1 + 0.9 * (-1) = -1.9$

► $V(0, 2) = \max(-1, -1.9, -1, -1.9) = -1$

En este punto, cualquier acción es igualmente buena. Arbitrariamente, elegimos Este. El barco se mueve a (0, 3).



ITERACIÓN DE VALOR

- ▶ **Paso 3 de 3:** Extracción de la Política Óptima.

- ▶ Una vez que la función de valor $V(s)$ ha convergido, podemos extraer la política óptima para cada celda navegable:

$$\pi(s) = \operatorname{argmax}_a [R(s, a) + \gamma * V(s')]$$

- ▶ Esto significa que para cada celda s , la política óptima $\pi(s)$ elige la acción a (Norte, Sur, Este, Oeste) que maximiza el valor esperado de la celda resultante s' .
- ▶ Ejemplo: Si después de la iteración de valor, encontramos que para una celda s :

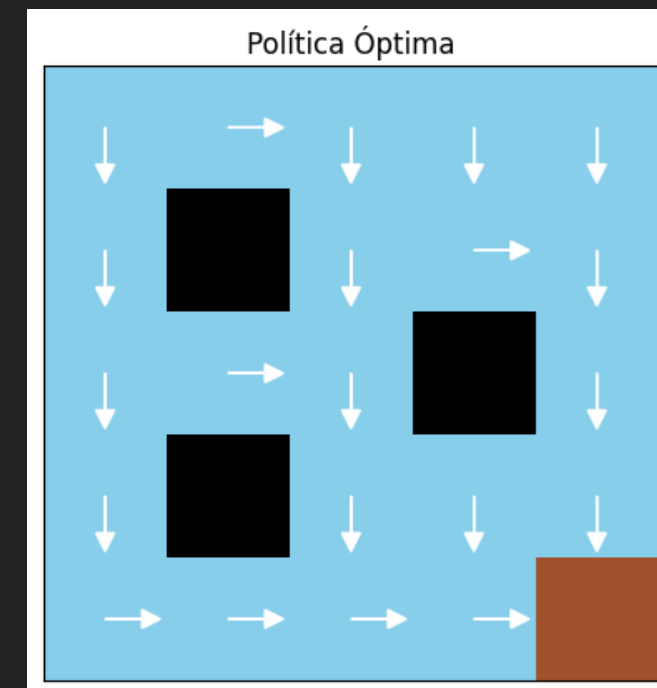
✓ $(-1 + 0.9 * V(\text{norte})) = -2$

✓ $(-1 + 0.9 * V(\text{sur})) = -3.5$

✓ $(-1 + 0.9 * V(\text{este})) = -1.5$

✓ $(-1 + 0.9 * V(\text{oeste})) = -1.8$

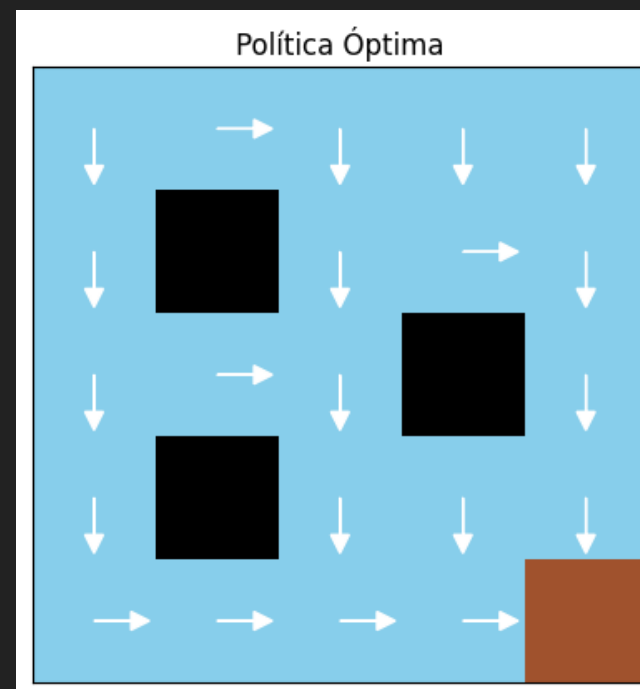
- ▶ Entonces, la **política óptima** para la celda s sería **ir al Sur** (porque **-1.5** es el valor más alto).



ITERACIÓN DE VALOR

► Observaciones

- ✓ Con cada iteración, los valores de los **estados (celdas)** se ajustan y se vuelven más negativos. Esto es porque estamos penalizando cada movimiento con una **recompensa de -1**.
- ✓ Después de varias iteraciones, la **política** (la acción que el barco elige en cada estado) **comenzará a converger hacia la ruta óptima al muelle**. Por ejemplo, el barco comenzará a aprender a evitar las rocas y a moverse hacia el este y el sur para llegar al muelle.
- ✓ Este proceso de iteración **continúa hasta que los valores de los estados dejen de cambiar** significativamente. En este punto, **hemos encontrado la función de valor óptima** y podemos extraer la **política óptima**.



ITERACIÓN DE VALOR

► Implementación en Python

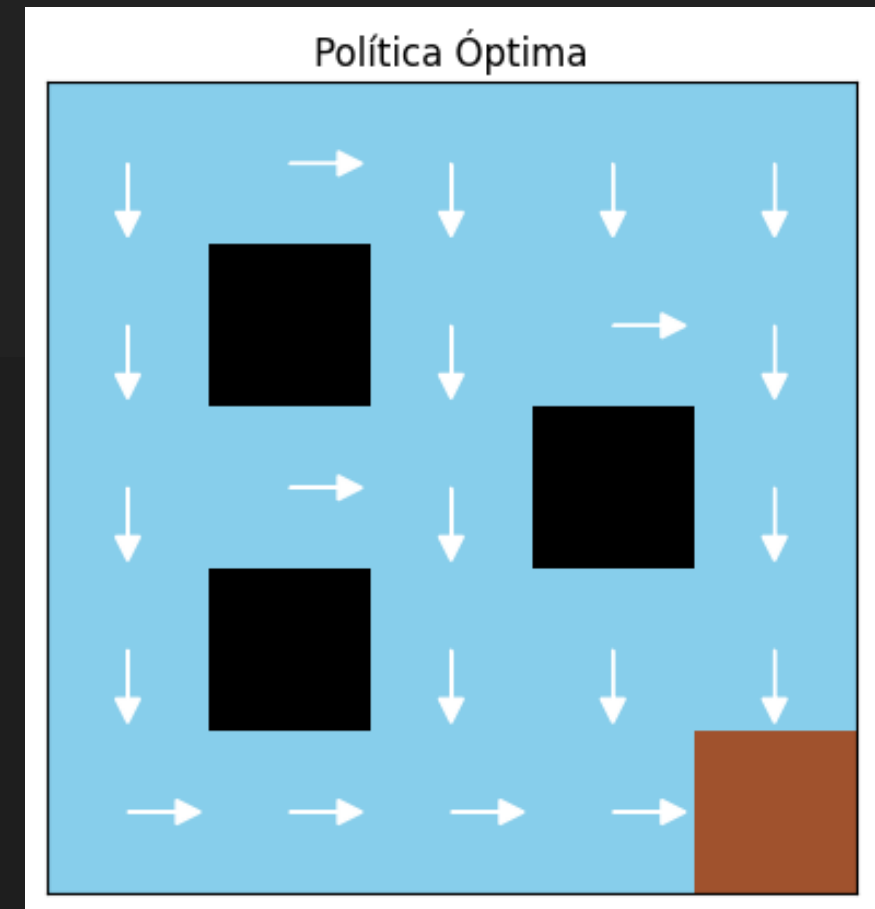
```
# algoritmo de iteracion de valor
iteracion = 0
while True:
    iteracion += 1
    delta = 0 # para verificar la convergencia

    # copia de la nueva finco de valor v(s)
    funcion_valor_anterior = funcion_valor.copy()

    for fila in range(LADO_OCEANO):
        for columna in range(LADO_OCEANO):
            if es_roca(fila, columna):
                continue # no se actualizan las rocas
            if (fila, columna) == MUELLE:
                continue # no actualiza el valor del muelle
            # aqui se calcula el valor de cada celda con la ecuación de Bellman
            valores_acciones = []
            for accion in ACCIONES:
                nueva_fila, nueva_columna = transicion(fila, columna, accion) # transicion me devuelve la nueva posicion de barco (nuevo estado)
                recompensa_inmediata = recompensa(fila, columna, accion) # recompensa devuelve 0: muelle, -1: avance, -100:colision con roca
                valores_acciones.append(recompensa_inmediata + FACTOR_DESCUENTO * funcion_valor[nueva_fila, nueva_columna]) # ecuacion de bellman

            funcion_valor[fila, columna] = max(valores_acciones)
            delta = max(delta, abs(funcion_valor[fila, columna] - funcion_valor_anterior[fila, columna]))

    print(f"Iteración {iteracion}: Delta = {delta}")
    # impresion poe consola del valor de cada celda en formato matricial
    for i in range(LADO_OCEANO):
        print(funcion_valor[i,:])
    if delta < TOLERANCIA:
        break
```



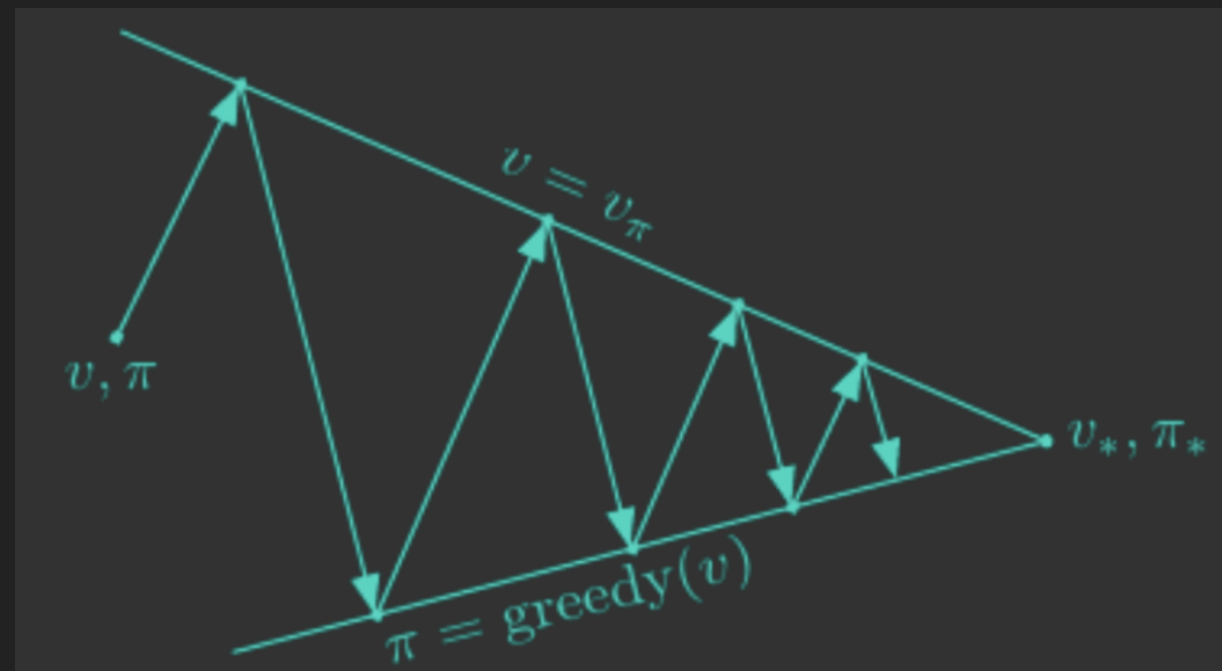
ITERACIÓN DE LA POLÍTICA GENERAL (GPI)

- ▶ Muchos de los algoritmos de **Aprendizaje por Refuerzo** comparten una estructura común (en general):
 1. Tienen un proceso para evaluar la "bondad" de la política actual (**policy evaluation**).
 2. Tienen un proceso para mejorar la política basándose en la evaluación anterior (**policy improvement**).
- ▶ El concepto de **GPI** establece que muchos algoritmos de RL (tales como **Iteración de Políticas**, **Iteración de Valor**, **SARSA**, **Q-Learning**, entre otros) pueden ser entendidos como **instancias que se caracterizan por la interacción de dos procesos: Evaluación de la Política y Mejora de la Política**.
- ▶ La forma en que se implementan estos procesos puede variar ampliamente, pero la idea central de iterar entre la **evaluación** y la **mejora** de la política sigue siendo la misma.



ITERACIÓN DE LA POLÍTICA GENERAL (GPI)

- ▶ La condición de parada se produce cuando la política no se puede mejorar al realizar los procesos de actualización.
- ▶ Si ambos procesos (Evaluación y Mejora) se estabilizan significa que v y π son óptimos.
- ▶ La función de valor v es estable cuando esta de acuerdo con la política actual y π es estable cuando es codiciosa (greedy) con respecto a v .



CONCLUSIONES

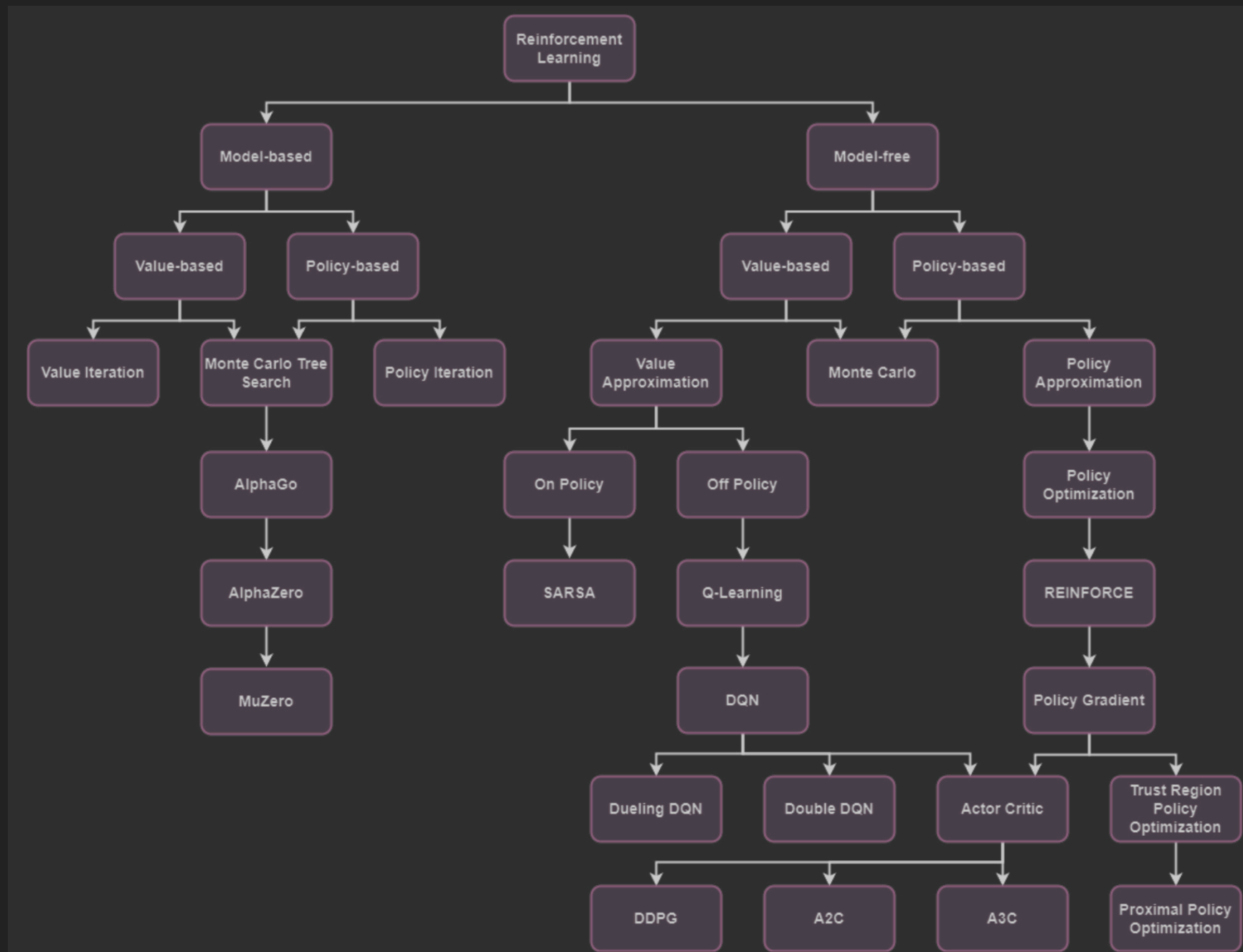
- ▶ Programación Dinámica (DP) se usa para resolver Procesos de Decisión de Markov (MDP).
- ▶ Iteración de Valor e Iteración de Políticas son algoritmos fundamentales y son la base de muchos algoritmos en Aprendizaje por Refuerzo (RL).
- ▶ Iteración de Valor e Iteración de Política son aplicados en áreas como control óptimo, robótica, planificación* y economía cuando el modelo es conocido.

* Implica usar un modelo de un entorno para analizar y mejorar decisiones antes de ejecutarlas en el entorno real.

CLASIFICACIÓN DE TÉCNICAS DE RL

- ▶ En RL decir **modelo** o **dinámica del entorno** son sinónimos.
- ▶ La **dinámica del entorno** es el conjunto de reglas que determinan cómo cambia el estado en función de las acciones y qué recompensas se obtienen.
- ▶ Un **modelo** describe las transiciones de estado: $p(s', r | s, a)$, que nos dice con qué probabilidad terminaremos en el **estado s'** y recibiremos la **recompensa r** tras tomar la **acción a** en el **estado s** .
- ▶ En RL tenemos 2 tipos de algoritmos:
 - ✓ **Model-Based** $\leftarrow$ (conozco $p(s', r | s, a)$)
 - ✓ **Model-Free** $\leftarrow$ (no conozco $p(s', r | s, a)$)

CLASIFICACIÓN DE TÉCNICAS DE RL



REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- ▶ R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- ▶ Khadivi, M., Charter, T., Yaghoubi, M., Jalayer, M., Ahang, M., Shojaeinasab, A., & Najjaran, H. (2025). Deep reinforcement learning for machine scheduling: Methodology, the state-of-the-art, and future directions. Computers & Industrial Engineering, 110856.
- ▶ N. Mazyavkina, S. Sviridov, S. Ivanov, and E. Burnaev. Reinforcement learning for combinatorial optimization: A survey. Computers & Operations Research, 134:105400, 2021.